

FUNCTIONAL GENOMICS · GRADUATE COURSE

功能基因组学

从 R 语言到组学：用代码打开功能基因组学的黑箱

功能基因组学 · 研究生课程

南方医科大学 基础医学院 生物信息学系

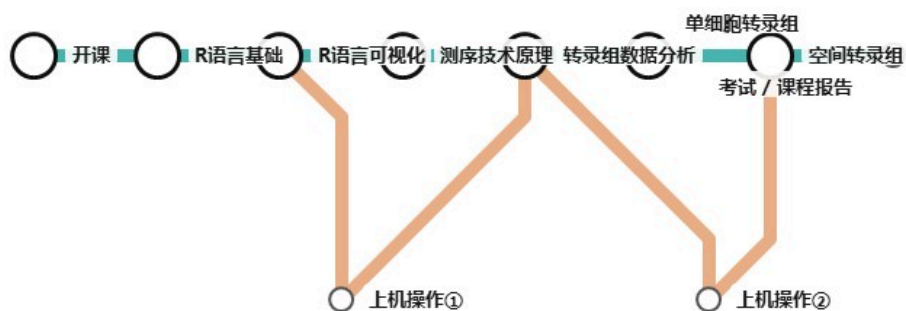
余光创

1 / 132

COURSE ROADMAP

这门课的旅程地图

五讲主线 + 两次上机操作，从工具走到真实组学数据



LEGEND

主线 · 5讲 + 考试

LAB · 两次上机

COURSE AGENDA

- **R语言**
工具篇：把环境装好，跑第一行代码
- **R语言可视化**
表达篇：让数据开口说话
- **测序技术**
原理篇：数据究竟是怎么来的
- **转录组数据分析**
基础篇：bulk RNA-seq 标准流程
- **单细胞转录组**
进阶篇：一个细胞一个转录组
- **空间转录组**
前沿篇：位置 + 表达
- **上机操作（两次）**
实践篇：两次真实数据动手
- **考试 / 课程报告**
成果篇：用报告证明你真的会了

2 / 132

FUNCTIONAL GENOMICS · LECTURE 01

R 语言

从零开始，把数据变成能复现的结论

功能基因组学 · 研究生课程 · 第一讲
南方医科大学 基础医学院 生物信息学系
余光创

3 / 132

PART 00

本讲地图

七个模块，一条从「装好」到「能交付」的完整路径

- 为什么是 R，而不是别的工具
- 先跑起来：安装、IDE、第一次交互
- 数据是什么：向量、列表、数据框
- 怎么操作数据：索引、赋值、向量化
- 控制流与函数：让机器替你重复
- 数据进出与可重复报告
- 从用包到写包

4 / 132

LEARNING OUTCOMES

学完这一讲，你应该能

目标不是「记住语法」，而是「能独立跑通一条最小分析链」

能装

在自己的电脑上装好 R 与 IDE，并知道装错了怎么查

能读

看懂一段陌生的 R 代码在做什么

能改

把别人的代码改成适合自己数据的版本

能查

遇到不认识的函数，知道去哪里找答案

能交

用 R Markdown 交出一份可复现的实验报告

一句话：从「抄代码」走到「改代码」，再到「自己写代码」

5 / 132

HOW TO USE

这份课件怎么用？

同一份材料，不同的用法决定学习效果

课前

按目录把 R 与 RStudio 装好 —— 环境问题留在课前解决

课上

重点听「为什么这么设计」，语法细节记不住没关系

课后

把每一页的代码亲手敲一遍，不要复制粘贴

卡住时

先读报错信息，再用 ?函数名 查文档，最后才来问

进阶

带着自己的数据来 —— 用真实问题驱动学习效率最高

课件是地图，不是终点。真正的学习发生在你自己敲代码的时候

6 / 132

WHY THIS MATTERS



会跑代码的研究生，和不会跑的，差的不是聪明，是能不能把想法变成可复现的证据。

— 本讲想传达的一句话

7 / 132

QUESTION · WHY NOT EXCEL?



"Let's not kid ourselves: The most widely used piece of software for statistics is Excel."

— Brian D. Ripley (R Core Team)

attributed keynote on statistical software adoption

8 / 132

FOLLOW-UP

那既然如此，我们为什么还要学代码？

承认 Excel 的现实地位，再谈它做不到的事；今天的目标是跨过那道门槛

不可复现

你做的每一步，都藏在鼠标点击里；换一个人，他不知道你点了什么

不可重跑

换一份数据，你要从头再点一遍；同样的错，要再排查一遍

不可审计

论文审稿人问「这个数怎么来的」，你拿不出一长串点击历史作为证据

代码 = 实验记录

代码本身就是完整的实验记录：能重跑、能审计、能交接、能迭代

所以本课的目标不是「学一门语言」，是「让分析变得可复现」

9 / 132

ICE BREAKER · 01 / 04

不会写代码

别人的数据集，看得到，动不了

会写 R

别人的数据集，是你发论文的原材料

10 / 132

ROADMAP

本讲路线图

每个模块都以「能做什么」结束，而不是以「讲了什么」结束



七步走完，你就具备了独立做一次生信分析的全部基础

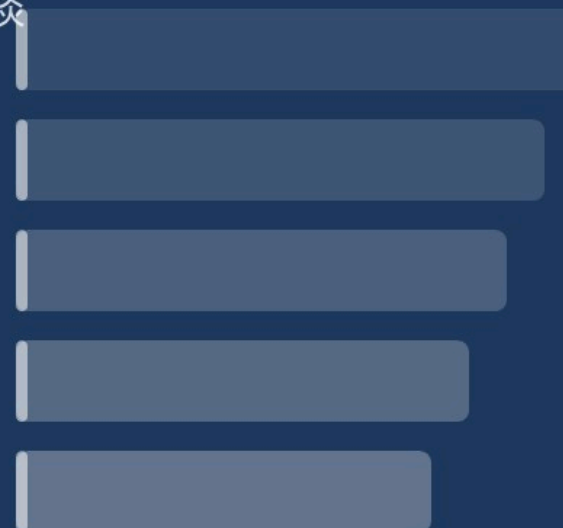
11 / 132

PART 01

先跑起来

把环境装好，跑出第一行代码 —— 这一步不完成，后面全是空谈

- R 是什么：一门语言 + 一个环境
- 为什么生物信息学选 R
- 装 R、装 IDE、第一次对话
- R 的生态：CRAN 与 Bioconductor



12 / 132

WHAT IS R

什么是 R?

R 是一个统计计算与图形的语言和环境

R 是一门语言

像英语一样，有自己的词汇（函数）和语法（表达式）

R 是一个环境

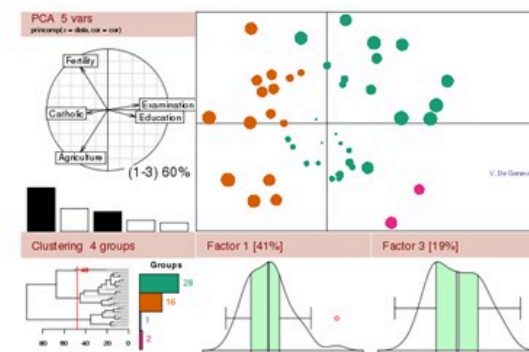
你输入的每一行，都会立刻被解释执行 —— 所见即所得

R 是一个社区

CRAN 上有两万多个包，几乎每个领域都有人替你写好了轮子

R 是一个标准

生物医学论文里的统计分析，绝大多数由 R 完成



R = Statistical Computing + Graphics, 一句话：为数据而生的语言

13 / 132

WHY R

为什么是 R?

在生物信息学这个场景下，R 有五个别人替代不了的理由

开源自由

免费、开放、可审计，学校和企业都能用

跨平台

Windows、macOS、Linux 上行为一致 —— 本地写的代码能直接上服务器

统计学的母语

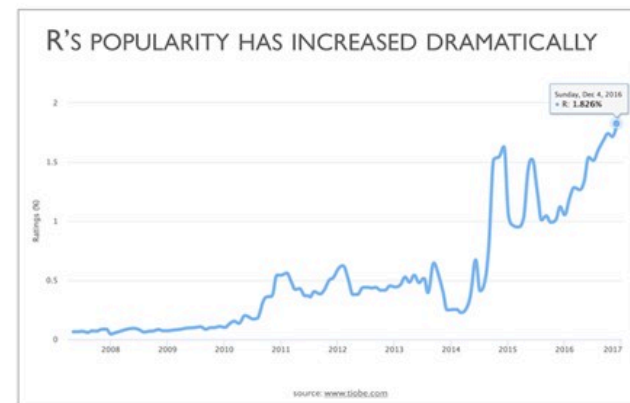
很多先进统计方法，只在 R 里才有实现

Bioconductor 生态

组学数据分析的事实标准社区，2000+ 专业包

图形能力

ggplot2 让「画图」从美术活变成可编程、可复现的工程



<https://www.sharpsightlabs.com/blog/r-recommend-data-science/>

14 / 132

CHOOSING A TOOL

R、Python、Excel：什么时候用哪个？

没有最好的工具，只有最合适的场景

R

- 统计建模与可视化最强
- Bioconductor 组学生态
- 交互式探索数据极快
- 写论文图表的第一选择

Python

- 通用编程，工程化更强
- 深度学习 / 大模型首选
- 生信工具有 scanpy 等
- 部署成服务更自然

本课选 R：因为我们要解决的是「分析数据、画出图、写进论文」

15 / 132

SETUP

第一步：把 R 装好

R 本体来自 CRAN —— Comprehensive R Archive Network

去哪下

搜索 CRAN，或直接访问 cran.r-project.org

选什么版本

选当前稳定版；Windows 下注意 32/64 位与 Rtools 的区别

装在哪

路径不要带中文和空格，否则后面装包会莫名失败

多版本工具

Rig (r-lib/rig) 一键装多版本 R；集群用 modules 切换版本

国内加速

换清华 / 中科大镜像，装包速度差一个数量级

The Comprehensive R Archive Network

Download and Install R

Precompiled binary distributions of the base system and contributed packages, **Windows and Mac** users most likely want one of these versions of R:

- [Download R for Linux \(Debian, Fedora/Redhat, Ubuntu\)](#)
- [Download R for macOS](#)
- [Download R for Windows](#)

R is part of many Linux distributions, you should check with your Linux package management system in addition to the link above.

装 R 只装一次，但镜像要设对 —— 这一步决定了后面顺不顺

16 / 132

安装与环境：原文链接

相关教程链接收在这里 —— 卡片本身可以直接点击打开

Windows 下新装 R 的极简指南

从零到跑通第一个脚本，完全没用过 R 的同学先看这篇

<https://mp.weixin.qq.com/s/rnk85VX9MZdmv-ITjEnh2w>



Windows 下多版本 R 与 Rtools 编译环境配置

要装多个 R 版本、或需要自己编译 R 包时看这篇

<https://mp.weixin.qq.com/s/zAXocRF1gcmZE2WUNqNUsA>



Linux 系统下多版本 R 安装

服务器与集群上最常用的做法

<https://mp.weixin.qq.com/s/uRBvgbV8YPe92JY6t36Xug>



Environment Modules: 集群软件环境控制

用 module avail / module load 在集群上切换软件版本

<https://github.com/envmodules/modules>



Rig: R 多版本一键管理 (r-lib)

macOS / Windows / Linux 通用，一条命令装齐 R release + devel

<https://github.com/r-lib/rig>

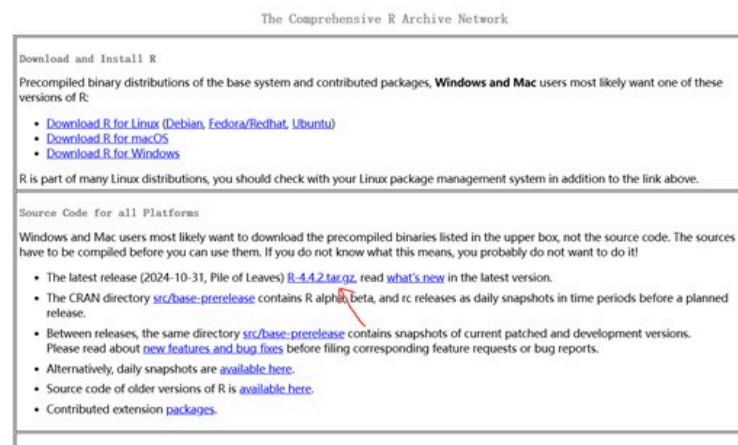


CRAN

CRAN: R 的中央仓库

所有官方包都从这里分发，装包就是从这里下载编译

- CRAN 上的包要经过审核，质量相对有保证
- 每个包都有文档、示例和测试
- 镜像站是完整拷贝，内容与官方一致，只是更快
- 设镜像: `options(repos = ...)` 指向国内镜像地址



CRAN 镜像站遍布全球，国内建议选清华 TUNA 或中科大 USTC

18 / 132

TOOLING

第二步：选一个顺手的 IDE

写 R 不一定要用 IDE，但用 IDE 会让效率提升一个台阶

RStudio / Positron

最主流的选择：编辑器 + 控制台 + 数据浏览器 + 文件管理器

为什么要 IDE

语法高亮、自动补全、变量面板、一键渲染报告

Positron

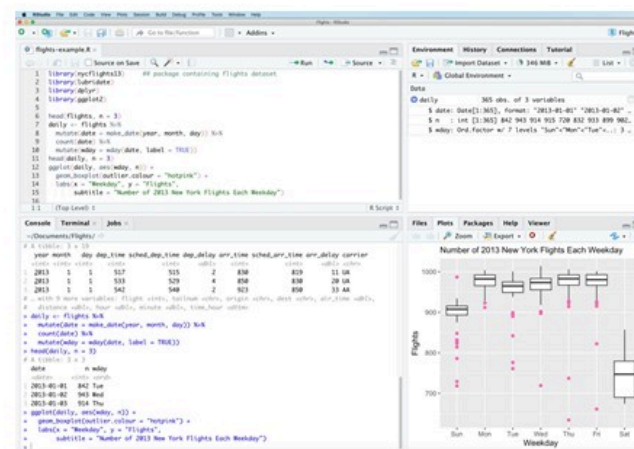
Posit 推出的新一代数据科学 IDE，对 R 与 Python 双支持

其他选择

VS Code + R 扩展、Jupyter + IRkernel、Emacs / Vim

一条建议

先用一个，用熟再换——频繁换工具是最浪费时间的



<https://mp.weixin.qq.com/s/wZtIOBGGN4eSKOtGqD6sqA>

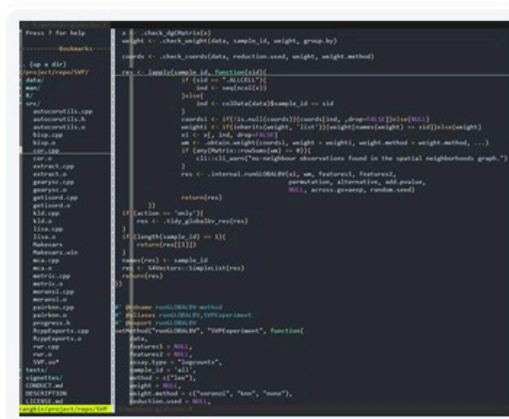
工具的选择不决定水平，但决定你每天要浪费多少时间在无关的事上

19 / 132

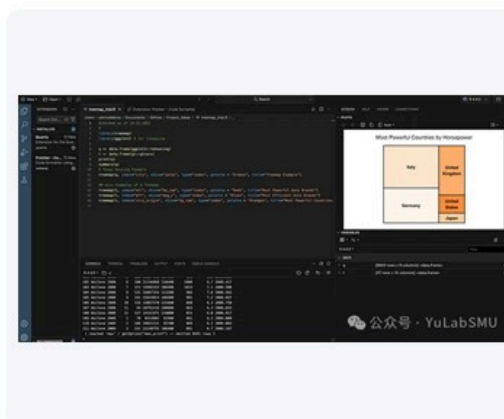
ENVIRONMENT

R 的三种最常用的打开方式

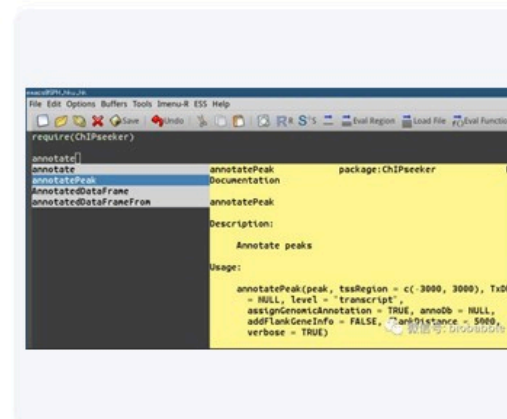
同一种语言，可以有很多种进入方式；下面是最常用的三种入口



VS Code + R 扩展：轻量、可定制



Jupyter Notebook：交互式、适合教学



Emacs / Vim：键盘流玩家的归宿

R语言的N种打开方式 <https://mp.weixin.qq.com/s/lblylRrD7mBD9acf52l7bw>

Rho: R 语言 AI 工作台 (R-centered Human-AI Orchestration)

YuLab-SMU 自研: 把持久 R 工作空间、项目感知编辑、科学结果面、AI 协作器塞进一个桌面应用

Rho 定位: Agent-native Workbench for R ↗

编辑器 / Console / AI Agent 共用同一个持久 R 工作空间 —— 你的操作、批准的 AI 动作, 都写进同一个 Workspace R session; Ask / Plan / Act

<https://github.com/YuLab-SMU/Rho>

本地化优先: 不依赖 Python / Jupyter / Electron ↗

三个平台原生桌面: Windows x64 / Apple Silicon macOS / Linux x86-64
; Rust + 自研 runtime, 支持多文档编辑、文件树、Console、环境面板、

<https://github.com/YuLab-SMU/Rho>

Rho 功能总览: 一个 R 语言 AI 工作台的全景 ↗

盘点 Rho 的核心能力: AI Ask / Plan / Act 三档协作、持久化 R Workspace 共享会话、Console / Editor / Plots / Environment 原生面板集成、R

https://mp.weixin.qq.com/s/A8PVvNPbC_b89x_HR-99GA

Rho 初体验: 5 分钟跑通第一个 R 工程 ↗

从下载安装 → 新建 R Project → 写脚本并运行 → 查看 Console / Environment / Plots 输出 → 触发一次 AI Ask 的完整流程; 全流程截图展示, 看完

<https://mp.weixin.qq.com/s/X7tY3J9UPRBoGNbVDIWWQ>

CHECKLIST

装好之后，做一次自检

这五行都能跑通，说明环境没问题

在控制台输入	期望看到	如果不对
<code>R.version.string</code>	版本号字符串	R 没装好，重装
<code>getOption("repos")</code>	一个 CRAN 镜像地址	镜像没设， <code>options(repos=...)</code>
<code>install.packages("ggplot2")</code>	开始下载安装	网络 / 镜像问题
<code>library(ggplot2)</code>	没有报错	包没装上，看报错信息
<code>sessionInfo()</code>	R 版本 + 包版本清单	正常，这就是你的环境快照

`sessionInfo()` 的输出贴给别人，对方就能复现你的环境

22 / 132

装完了？来，立刻抽个奖！

把下面 4 行代码粘进 RStudio 的 Console，敲回车 —— 你会抽到什么？



01

一等奖 · 彩虹出来了

`barplot(1:7, col=rainbow(7), main="今天运气不错！")` 一张 7 色柱子图，证明你的 R + 图形引擎都装对了。



02

二等奖 · 今天运气值

`cat("今日幸运值: ", sample(60:99,1), "/100\n")` 60 起步，绝对不会翻车，给你今天第一点正反馈



03

遇到红色 Error 了？

如果 Console 里出现 红色的 Error 信息：别慌。90% 的概率是 R 版本 / Rtools 没装对；安装卡片页有完整指南。

一句话：装完不是结束 —— 跑一行真正出结果的代码，才算真正开始。

23 / 132

READ THE CONSOLE

控制台上的三个符号

刚才你敲的每一行，R 都是这样读的

```
R console
> 1 + 1
[1] 2
> x <- 3 * 4
> x
[1] 12
> q()
```

- ▮ `>` 是命令提示符：它后面才是你输入的内容
- ▮ [1] 是输出行号，表示这是结果的第 1 个元素
- ▮ <- 是赋值符号，把右边的值存进左边的名字里
- ▮ q() 退出 R —— 注意它不会自动保存你的工作

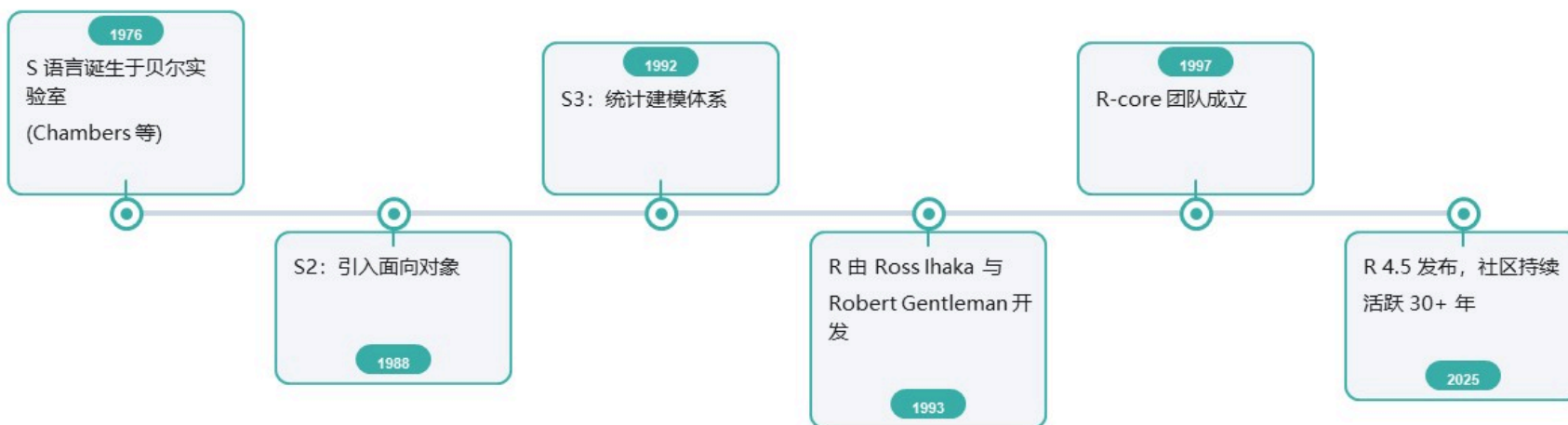
记住三个符号：> 是提示，<- 是赋值，() 是调用

24 / 132

HISTORY

R 从哪来?

R 是 S 语言的开源实现 —— 青出于蓝，而胜于蓝



"S has forever altered the way people analyse, visualise and manipulate data."

— John Chambers, one of the originators of S at Bell Labs

"R is the most powerful and flexible statistical programming language in the world."

— Norman Nie, scholar and co-founder of SPSS

25 / 132

ECOSYSTEM

R 应用有多广?

CRAN Task View 按领域整理了所有包，从统计到生态到医学影像

- 两万多个包，按 40+ 个任务视图分类

- 生物信息学是最活跃的领域之一

- 大多数包有论文支撑，可引用

- 找包先查 Task View，别从零开始写

Topics

[ActuarialScience](#)
[Agriculture](#)
[Bayesian](#)
[CausalInference](#)
[ChemPhys](#)
[ClinicalTrials](#)
[Cluster](#)
[Databases](#)
[DifferentialEquations](#)
[Distributions](#)
[Econometrics](#)
[Environmetrics](#)
[Epidemiology](#)
[ExperimentalDesign](#)
[ExtremeValue](#)
[Finance](#)

Actuarial Science
Agricultural Science
Bayesian Inference
Causal Inference
Chemometrics and Computational Physics
Clinical Trial Design, Monitoring, and Analysis
Cluster Analysis & Finite Mixture Models
Databases with R
Differential Equations
Probability Distributions
Econometrics
Analysis of Ecological and Environmental Data
Epidemiology
Design of Experiments (DoE) & Analysis of Experimental Data
Extreme Value Analysis
Empirical Finance

来源: CRAN Task Views — cran.r-project.org/web/views/

26 / 132

BY THE NUMBERS

R 生态的三个数字

这些数字背后，是你可以直接拿来用的现成工具

20,000+

CRAN 上的包

覆盖统计、机器学习、可视化、报告自动化

2,300+

Bioconductor 包

组学数据分析的事实标准社区

30+

年持续维护

1993 年至今，R-core 团队持续发布

数据来源：CRAN / Bioconductor 官网，截至 2026 年

27 / 132

ECOSYSTEM

Bioconductor: 生信人的 R

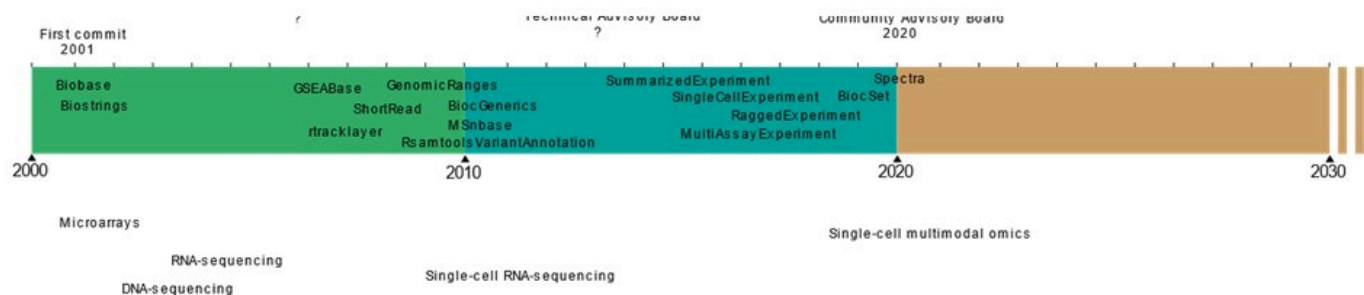
2001 年启动，专门为组学数据分析而建

- 按季度发布，与 R 版本严格对应

- 实验数据包、注释包、软件包三类并存

- 同一个包有 release 与 devel 两条线

- 用 `BiocManager::install()` 安装，不要用 `install.packages()`



bioconductor.org — 有独立的发布周期与审核标准，比 CRAN 更严

28 / 132

WORKFLOW

交互式 vs 脚本：两种工作模式

新手常犯的错，是只用第一种

交互式（控制台）

- 想到什么敲什么，立刻看结果
- 适合探索数据、试错
- 关掉就没了，不可复现
- 适合：搞清楚一件事怎么做

脚本（.R 文件）

- 一次写好，可以反复运行
- 可以保存、分享、版本管理
- 改数据后重跑，结果自动更新
- 适合：真正的分析工作

正确姿势：在控制台里探索，探索通了就写进脚本

29 / 132

WHY R

R 的设计哲学

同样一件事，R 的写法往往更接近数学表达

向量化

对一整列数据做运算，不用写循环

公式语法

$y \sim x$ 直接描述统计模型，可读性极高

图形语法

把图拆成数据 + 映射 + 几何对象，可以组合

函数式风格

apply 家族、管道操作，让数据变换像流水线

统计原生

分布函数、假设检验、回归模型都是内置的

R 的设计目标：让统计学家写代码时，脑子里想的是统计而不是编程

30 / 132

SUMMARY

PART 01 小结

环境准备好了，接下来进入真正的核心：数据

装好

R 本体 + 一个 IDE + 国内镜像

认清

R = 语言 + 环境 + 生态

会用

提示符、赋值、函数调用三种基本动作

知道去哪找

CRAN 找通用包, Bioconductor 找组学包

下一步：R 里到底有几种数据？为什么说「一切都是向量」？

31 / 132

数据是什么

理解 R 的数据结构，是理解 R 一切行为的前提

- 三种基本类型：数值、字符、逻辑
- 向量：R 的最小积木
- 列表：什么都能装
- 矩阵、数组、数据框：积木的搭法
- 缺失值 NA / NULL / NaN
- 属性与面向对象：为什么 R 这么灵活

CORE IDEA

一句话理解 R 的数据

把复杂结构拆开看，底层都是它



抓住「向量」这个概念，R 里 80% 的「奇怪行为」都变得合理

33 / 132

BASE TYPES

三种基本类型

所有复杂结构，都是这三种原子类型搭出来的

数值型 numeric

1, 3.14, -0.5 —— 默认是双精度浮点型 (double), 写 1L 才是整数型

字符型 character

用引号包起来的就是字符串，单引号双引号都可以

逻辑型 logical

TRUE / FALSE —— 用于判断，也可以参与算术 (TRUE 当 1)

类型决定了这个数据能做什么运算 —— 类型错了，结果就错了

34 / 132

DETAIL

数值型的两个坑

看起来都是「数字」，但 R 内部并不一样

```
> storage.mode(1)
[1] "double"
> storage.mode(1L)
[1] "integer"
> complex(real = 1, imaginary = 2)
[1] 1+2i
```

- 默认数字是 double (双精度)，不是 integer
- 写 1L 或 as.integer() 才能得到整数型
- 整数运算更快、更省内存，大数据时值得注意
- R 还支持复数型 complex，工程计算里会用到

日常分析不用纠结，但要知道 1 和 1L 不是同一个东西

35 / 132

DETAIL

字符型：不止是文本

R 内置正则表达式，字符串处理能力很强

```
> pkg <- rownames(installed.packages())
> grep("\\d$", pkg, value = TRUE)
[1] "bit64"    "cpp11"    "DESeq2"
[4] "e1071"    "ggplot2"  "lme4"
[7] "R6"       "xml2"     "stats4"
```

- ▀ \d 匹配数字, \$ 匹配字符串结尾
- ▀ grep() 返回匹配位置, value=TRUE 时返回内容
- ▀ 正则表达式是处理基因名、样本名、注释表的利器
- ▀ 不会写正则? ?regex 有完整语法表

字符串处理是生信分析中最常被低估的技能

36 / 132

DETAIL

逻辑型：判断的结果

逻辑型是「筛选」的基础 —— 几乎所有子集操作都靠它

```
> 1:3 > 2
[1] FALSE FALSE TRUE
> sum(1:3 > 2)
[1] 1
> which(1:3 > 2)
[1] 3
```

- 比较运算对向量逐元素进行，返回等长的逻辑向量
- 逻辑向量可以直接求和：TRUE 计为 1，FALSE 计为 0
- which() 把逻辑向量转成位置下标
- 这就是「条件筛选」的底层机制

记住：筛选 = 先算出一个逻辑向量，再用它取子集

37 / 132

VECTOR

向量：R 的最小积木

向量化运算是 R 最核心的语法特征

```
> 1:3
[1] 1 2 3
> 1:3 * 3
[1] 3 6 9
> 1:3 * 3 + 2
[1] 5 8 11
> 1:3 * 1:3
[1] 1 4 9
```

1:3 生成 1,2,3 的整数向量

整列乘 3 —— 不需要写循环，R 自动逐元素运算

长度相同的两个向量可以逐元素运算

长度不同时「循环补齐」，这是常见的隐藏 bug 来源

向量化 = 用一行代码代替一个循环，这是 R 快的原因之一

38 / 132

VECTOR

循环补齐：一个安静的陷阱

长度不一致时 R 不报错，而是默默重复短的向量

```
> 1:6 + c(10, 20)
[1] 11 22 13 24 15 26

> 1:5 + c(10, 20)
[1] 11 22 13 24 15
Warning message:
longer object length is not a
multiple of shorter object length
```

- 短向量会被重复使用，直到补齐长向量的长度
- 长度成整数倍时完全不报警 —— 这才是最危险的
- 出现 Warning 说明大概率不是你想要的结果
- 看到这类警告，先检查两个向量的长度

养成习惯：做向量运算前，`length()` 检查一下两边长度

39 / 132

MISSING

NA、NULL、NaN: 三个「空」的区别

这是新手最容易混淆的一组概念

	NA	NULL	NaN
含义	缺失值 (位置还在)	不存在 (位置也没了)	非数字 (0/0)
长度	占 1 个位置	长度 0	占 1 个位置
is.na()	TRUE	logical(0)	TRUE
is.null()	FALSE	TRUE	FALSE
典型场景	数据表里的空单元格	列表里删掉元素	非法数学运算

记住: NA 是「有位置但没值」, NULL 是「连位置都没有」

40 / 132

MISSING

缺失值怎么处理?

很多「莫名其妙」的结果，根源都是 NA

```
> x <- c(1, 2, NA, 4)
> mean(x)
[1] NA
> mean(x, na.rm = TRUE)
[1] 2.333333

> is.na(x)
[1] FALSE FALSE TRUE FALSE
> x[!is.na(x)]
[1] 1 2 4
```

看到统计结果是 NA，第一反应应该是「是不是有缺失值」

- 只要有一个 NA，默认统计函数就返回 NA
- na.rm = TRUE 让函数跳过缺失值
- 判断是否缺失用 is.na()，不要用 x == NA（永远返回 NA）
- 取子集时 !is.na(x) 是最常用的清洗写法

41 / 132

TYPES

类型强制转换

R 会按「逻辑 < 整数 < 数值 < 字符」的优先级自动升级

```
> c(1, "a")
[1] "1" "a"      # 全变字符
> c(TRUE, 2)
[1] 1 2        # TRUE 变 1
> c(1L, 2.5)
[1] 1.0 2.5     # 全变 double

> as.character(1:3)
[1] "1" "2" "3"
```

向量的所有元素类型必须一致，所以会自动向上取「最宽」的类型

这个机制叫 coercion，是很多「结果不对」的根源

显式转换用 `as.numeric` / `as.character` / `as.logical`

读数据时把数值列读成字符，后面做运算就会报错

一个向量里混了数字和字符串，数字会变成字符串 —— 检查 `str()` 最保险

42 / 132

TYPES

类型判断与转换速查

判断用什么，转换用什么，一张表说清

判断	转换	说明
<code>is.numeric(x)</code>	<code>as.numeric(x)</code>	数值型
<code>is.character(x)</code>	<code>as.character(x)</code>	字符型
<code>is.logical(x)</code>	<code>as.logical(x)</code>	逻辑型
<code>is.factor(x)</code>	<code>as.factor(x)</code>	因子
<code>is.na(x)</code>	—	是否缺失（最常用）
<code>class(x)</code>	—	看对象的类（最通用）
<code>str(x)</code>	—	看完整结构（最推荐）

不确定对象是什么，`str(x)` 一次看清 —— 这是最重要的调试习惯

43 / 132

FACTOR

因子型：有顺序的字符

因子是 R 里最容易被误解的类型，但统计建模离不开它

```
> y <- factor(letters[1:3], levels = letters[3:1])
> y
[1] a b c  Levels: c b a
> as.numeric(y)
[1] 3 2 1
> sort(y)
[1] c b a  Levels: c b a
```

因子 = 字符 + 水平(levels), levels 决定顺序

as.numeric() 拿到的是「水平编号」，不是原值

统计模型把因子当作分类变量处理

画图时因子顺序决定坐标轴顺序 —— 经常需要手动设 levels

因子最常见的用途：控制分组在图表上的排列顺序

44 / 132

FACTOR TRAP

因子的经典陷阱

这一个坑，几乎每个人都踩过

```
> x <- factor(1:3, levels = 3:1)
> x
[1] 1 2 3  Levels: 3 2 1
> as.numeric(x)
[1] 3 2 1      # 错! 拿到的是编号
> as.numeric(as.character(x))
[1] 1 2 3      # 对: 先还原字符再转数值
```

as.numeric(factor) 返回水平编号，不是原始数值

正确姿势：as.numeric(as.character(x))

读入数据时用 stringsAsFactors = FALSE 可避免自动转因子

R 4.0 之后默认不再自动转因子，老代码要注意

看到 as.numeric() 作用在因子上，先停下来想一秒

45 / 132

LIST

列表：什么都能装

如果说向量是「同质的一排」，列表就是「贴着标签的收纳箱」

```
> y <- list(a = 1:3, b = LETTERS[1:2], c = rnorm(3))
> y$a
[1] 1 2 3
> y[["b"]]
[1] "A" "B"
> names(y)
[1] "a" "b" "c"
```

列表的元素可以是任意类型：向量、矩阵、数据框、函数

数据框本质上就是一个列表 —— 每列是一个向量

很多函数的返回值都是列表，所以必须学会取列表元素

命名后可以用 `$` 或 `[["名字"]]` 取内容

理解列表，就理解了 R 为什么能装下任意复杂的分析结果

46 / 132

MATRIX

矩阵：加了维度属性的向量

矩阵不是新东西 —— 它就是一个向量，外加一个 dim 属性

```
> x <- 1:6
> dim(x) <- c(2, 3)
> x
      [,1] [,2] [,3]
[1,]    1    3    5
[2,]    2    4    6
> x[1, 3]      # 第1行第3列
[1] 5
```

矩阵 = 向量 + dim 属性，所以它仍然可以当向量用

默认按列填充：1,2 是第一列，3,4 是第二列

矩阵里所有元素必须是同一类型

表达矩阵、距离矩阵、邻接矩阵都是矩阵

属性(attribute) 是 R 用简单结构搭出复杂结构的关键机制

47 / 132

ATTRIBUTE

属性：R 的元数据机制

数据可以携带关于自己的信息，这就是属性

```
> x <- letters[1:3]
> x <- structure(x, abc = "hello")
> attributes(x)
$abc
[1] "hello"
> names(x) <- c("p", "q", "r")
> str(x)
Named chr [1:3] "a" "b" "c"
```

属性是附加在数据上的元数据，不改变数据本身

names、dim、dimnames、class 都是属性

str() 是查看对象结构的最佳工具

R 对属性几乎没有限制 —— 这也是它能自由扩展的原因

记住 str(): 它是你搞清楚「这个对象到底是什么」的第一工具

48 / 132

DATA FRAME

数据框：最常用的结构

数据框 = 特殊的列表：每列是等长向量，列内同质，列间可异质

```
> df <- data.frame(  
  gene = c("TP53", "EGFR", "MYC"),  
  logFC = c(2.1, -1.5, 0.8),  
  sig = c(TRUE, TRUE, FALSE)  
)  
> df$logFC  
[1] 2.1 -1.5 0.8  
> nrow(df); ncol(df)  
[1] 3  
[1] 3
```

数据框是 90% 分析工作的主战场，必须熟练

- 每一列是一个向量，所以列内类型必须一致
- 列间类型可以不同 —— 这是它比矩阵强的地方
- 表达矩阵 + 样本信息表，是生信分析最常见的组合
- tibble 是数据框的现代版本，打印更友好、限制更严格

49 / 132

OVERVIEW

R 的数据结构全景

从最简单到最复杂，一条清晰的路

向量 vector

一维、同质。1:3、c("a","b")。R 的最小积木

矩阵 / 数组

多维、同质。矩阵是二维，数组可以更高维

列表 list

一维、异质。可以装下任何东西，包括函数

数据框 data.frame

列表的特例：等长列向量。表格数据的标准容器

因子 factor

字符 + 顺序。用于分类变量

S3 / S4 / R6 对象

面向对象的容器。Bioconductor 大量使用 S4

先掌握向量、列表、数据框三件套，其余按需再学

50 / 132

OBJECT ORIENTED

R 的面向对象：为什么重要？

Bioconductor 的包几乎都是面向对象设计 —— 看不懂 S4，就读不懂源码

S3：最简单的实现

本质是「带 class 属性的列表」。历史悠久、实现简单、执行快

S3 的短板

没有明确的类定义和检查机制，写错了不会报错

S4：严谨的升级版

有明确的类定义、数据槽(slot)定义和初始化检查

R6 / 引用类

支持可变状态，适合写有状态的程序

为什么 R 用得多

S4 让 Bioconductor 项目在多人协作下保持稳健

你不需要马上会写 S4，但必须能看懂 setClass / setMethod

51 / 132

R 与 C++ 的面向对象差别

同样是面向对象，R 的思路完全不同 —— 这正是它「看起来像面向过程」的原因

C++：以类为核心

- 类定义数据 + 操作方法
- 调用语法：对象.方法()
- 方法是类的一部分
- 新增行为需要改类

R：以方法为核心

- 类只定义数据
- 调用语法：方法(对象)
- 泛型函数 + 方法定义在类之外
- 不改类也能为已有类加新方法

所以 R 代码看着像「面向过程」，实际是「泛型函数分派」

SUMMARY

PART 02 小结

数据结构是地基 —— 地基牢，后面才稳

三件套

向量（同质一维）、列表（异质）、数据框（表格）

一个关键机制

属性让简单结构能搭出复杂结构

一个必会工具

`str()` 看结构, `class()` 看类型, `length()` 看长度

两个常见坑

因子转数值要先 `as.character()`; NA 会污染统计结果

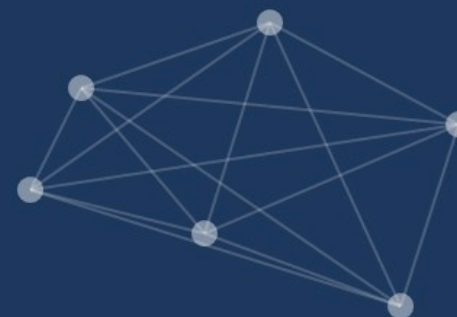
下一问：知道了数据长什么样，怎么把它取出来、改掉？

53 / 132

怎么操作数据

取子集、改内容、做运算 —— 分析工作 90% 的时间都花在这里

- 赋值: `<-` 与 `=` 的区别
- 取子集: `[]`、`[[]]`、`$`
- 命名与键值查找
- 修改与删除
- `apply` 家族与管道: R 的流水线
- 一个颠覆认知的事实: 运算符也是函数



SYNTAX

赋值：<- 还是 = ？

这是一个 R 社区争论了三十年的问题

历史原因

S 语言的键盘上真的有 <- 这个键，所以沿用了下来

推荐做法

赋值用 <-，函数传参用 =，这样代码意图最清楚

等价情况

在顶层赋值时 $x <- 1$ 与 $x = 1$ 效果相同

不等价情况

函数调用里 $x = 1$ 是「传参」，不是赋值，语义完全不同

<- 的方向性

$x <- 1$ 读作「把 1 放进 x」，也可以写成 $1 \rightarrow x$



微信号: biobabble

R语言冷知识，为什么赋值要用 <- https://mp.weixin.qq.com/s/nXZBgogskN3FdacYD6M_gg

养成习惯：赋值一律用 <-，传参一律用 =，读代码时不会看错

55 / 132

SYNTAX

运算优先级

数值运算 > 关系运算 > 逻辑运算 —— 记不住就加括号

```
> 2 == 5 - 4 | 3 > 2  
[1] TRUE
```

实际执行顺序:

# 1. 5 - 4	-> 1	数值运算
# 2. 2 == 1	-> FALSE	关系运算
# 3. 3 > 2	-> TRUE	关系运算
# 4. FALSE TRUE	-> TRUE	逻辑运算

优先级顺序: 数值 > 关系 > 逻辑

这个例子不加括号能跑, 但读起来很费劲

建议: 只要一行里混了两种以上运算, 就加括号

括号不影响性能, 只提升可读性

代码是写给人看的, 顺便让机器执行

56 / 132

SYNTAX

逻辑运算：| 与 || 的区别

这是 R 里最容易写错、也最难发现的地方

```
> c(TRUE, FALSE) | c(TRUE, TRUE)
[1] TRUE TRUE      # 向量化
> c(TRUE, FALSE) || c(TRUE, TRUE)
[1] TRUE           # 只看第一个元素

# if 语句里要的是单个值:
> if (c(TRUE, FALSE)) "yes"
[1] "yes"          # 只判断了第一个
```

- | 和 & 是向量化的，逐元素运算
- || 和 && 只比较第一个元素，返回单个 TRUE/FALSE
- if() 的条件必须是长度 1 的逻辑值
- 在 if 里用 | 可能不会报错，但结果不是你想要的

判断条件用 || / &&，向量筛选用 | / &

57 / 132

互动猜一猜：x[2:3] 到底返回什么？

假设 `x <- c(10, 20, 30, 40, 50)`。以下 3 种写法，选 1 个答案 —— 然后自己跑一遍验证。



A

x[c(2,3)] 取第 2、3 个

结果 = 20 30 正下标 → 要哪些 拿哪些，最符合直觉。



B

x[-2:3] 排除第 2~3 个？

哦不！R 会先算 `-2:3 = (-2, -1, 0, 1, 2, 3)`，然后 0 被丢弃，最终拿到 10 20 30。（想排除 2、3？请写 `x[-(2:3)]`）



C

x[x > 25] 按条件拿

内部先算 `x > 25 = F, F, T, T, T` 再用这 5 个布尔去筛：结果 = 30 40 50。这就是 R 的「向量化过滤」。

一句话：方括号是 R 的第一生产力 —— 写错方括号，数据再好也出不了图。

58 / 132

SUBSET

取子集：[] 的四种用法

中括号是 R 里使用频率最高的符号

```
> x <- c(10, 20, 30, 40)
> x[2]           # 按下标
[1] 20
> x[c(1, 3)]     # 按多个下标
[1] 10 30
> x[-1]          # 排除
[1] 20 30 40
> x[x > 15]      # 按条件
[1] 20 30 40
```

下标从 1 开始，不是 0 —— 这是 R 与多数语言的区别

负下标表示「排除」，不能和正下标混用

逻辑向量筛选是最常用、也最强大的方式

返回的是新对象，原对象不变（R 默认值语义）

记住：R 从 1 开始数，而且筛选就是「逻辑向量 + 中括号」

59 / 132

SUBSET

矩阵与数据框取子集

用两个下标：逗号前是行，逗号后是列

```
> m[1, ]      # 第 1 行
> m[, 2]      # 第 2 列
> m[1, 2]     # 第 1 行第 2 列
> m[-1, ]     # 去掉第 1 行

# 数据框按列名取
> df[, "logFC"]
> df$logFC
> df[df$sig, ] # 只留显著的
```

逗号留空表示「全部」，这是最常用的写法

矩阵取单列会退化成向量，加 `drop = FALSE` 可保持矩阵

数据框按列名取比按下标取更安全

`df[条件, ]` 是最经典的数据框筛选模式

`df[条件, ]` 这个模式，你会在接下来的三年里用上几千次

60 / 132

SUBSET

列表取值: [] 与 [[]] 与 \$

三个符号, 三种含义 —— 混淆它们是最常见的错误

```
> y <- list(a = 1:3, b = "x")
> y[1]      # 取「子列表」
$a
[1] 1 2 3
> y[[1]]    # 取「内容」
[1] 1 2 3
> y$a       # 按名字取内容
[1] 1 2 3
```

- 单括号 [] 返回列表的子集, 类型仍是 list
- 双括号 [[]] 把内容取出来, 类型是内容本身的类型
- \$ 等价于 [[]"名字"], 但不能用于向量
- 取不到时 [] 返回 NULL, [[]] 会报错 —— 调试时很有用

一句话: [] 是「切一块」, [[]] 是「拿出来」

61 / 132

SUBSET

命名：把向量当字典用

给向量命名之后，它就变成了一个有序的键值表

```
> x <- c(1, 2, 3)
> names(x) <- c("a", "b", "c")
> x["b"]
b
2
> x[c("a", "c")]
a c
1 3
```

names() 既可以读也可以写

按名字取子集比按下标更安全 —— 数据顺序变了也不会错

很多函数的返回值自带 names，方便按名取用

环境(environment) 也是键值容器，但它是无序的

能用名字取，就不要用下标取 —— 这是可维护代码的标志

62 / 132

MODIFY

修改与删除

取子集和赋值组合起来，就是修改

```
> x <- c(a = 1, b = 2, c = 3)
> x[2] <- 40      # 修改
> x
  a  b  c
1 40  3

> y <- list(a = 1, b = 2)
> y$b <- NULL     # 删除
> names(y)
[1] "a"
```

R 默认是值语义 —— 想清楚「我改的是原对象还是副本」

- `x[i] <- value` 就是「把第 *i* 个位置换成新值」
- 列表里把元素赋值为 `NULL` 就是删除该元素
- 向量不能直接删除元素，只能生成一个不含该位置的新向量
- 这就是「值语义」：修改通常产生新对象，而不是改原地

63 / 132

POWER TOOL

apply 家族：不用写循环

这是 R 里最实用、也最值得花时间掌握的一组函数

```
> apply(m, 1, sum)      # 对矩阵每行求和
> apply(m, 2, mean)     # 对每列求均值
> lapply(lst, length)   # 对列表每元素求长度，返回列表
> sapply(lst, length)   # 同上，但尽量简化为向量
> tapply(v, g, mean)    # 按分组 g 对 v 求均值
```

- apply: 对矩阵的行或列; 1 表示行, 2 表示列
- lapply: 输入列表, 输出列表 —— 最通用
- sapply: 结果能简化就简化, 方便但有时类型不可控
- tapply: 分组汇总, 等价于 aggregate 的轻量版

把「循环 + 累积」换成 apply 家族, 代码会短一半、快一倍

64 / 132

管道：把嵌套变成流水线

R 4.1 起内置了 `|>`，tidyverse 用 `%>%`

```
# 嵌套写法：从里往外读
round(mean(abs(c(-1.5, 2.5, -3))), 2)

# 管道写法：从上往下读
c(-1.5, 2.5, -3) |>
  abs() |>
  mean() |>
  round(2)
```

`|>` 是 R 4.1 起的内置管道，不需要装包

`%>%` 来自 `magrittr`，功能更多（如占位符 `.`）

管道把「先做什么、再做什么」按阅读顺序排列

复杂的数据变换用管道写，可读性提升非常明显

超过两层嵌套就该考虑用管道 —— 代码是给人读的

INSIGHT

颠覆认知：运算符也是函数

R 里没有「特殊语法」，一切都统一为函数调用

```
> 1 + 2
[1] 3
> `+`(1, 2)
[1] 3
> letters[1:3][2]
[1] "b"
> `[`(letters[1:3], 2)
[1] "b"
```

中缀运算符 + 其实就是名字叫 + 的函数

反引号 ` 可以把非标准名字变成函数名

所以「运算符」「操作符」「函数调用」在 R 里是同一件事

理解这一点，看 R 源码就不再神秘

R 的统一性：语法糖背后都是函数，函数背后都是方法

66 / 132

INSIGHT

泛型函数：同名不同行为

同一个 `plot()`，对不同的对象画出完全不同的图

```
> x <- table(sample(letters[1:4], 50, TRUE))
> class(x)
[1] "table"
> plot(x) # 画频次条形图
> plot(sin, -pi, 2*pi) # 画函数曲线
```

`plot()` 是泛型函数，它会根据第一个参数的类型选择方法

实际调用的是 `plot.table()`、`plot.function()` 等

用 `methods(plot)` 可以查看有哪些方法

这就是 R 面向对象的核心机制：泛型函数分派

看到函数行为「不符合预期」，先查 `class()` —— 可能是方法分派

67 / 132

INSIGHT

ggplot2 里的 + 号

同一个加号，在 ggplot2 里被重载成了「图层叠加」

```
> library(ggplot2)
> ggplot(mtcars, aes(wt, mpg)) +
  geom_point() +
  geom_smooth()
```

+ 在这里不是加法，是「往图上加一层」

ggplot2 为 + 定义了针对 ggplot 对象的方法

所以 + 号的行为取决于左边的对象类型

这也是初学者最容易困惑的点：为什么 + 不能乱加

自定义类也可以重载运算符 —— 这是 R 灵活性的来源

运算符行为由对象类型决定 —— 这是 R 的强大，也是它的陷阱

68 / 132

SUMMARY

PART 03 小结

数据操作的三个层次：取出来、改掉、算出来

取子集

`[]` 切一块, `[[ ]]` 拿内容, `$` 按名字拿

修改

取子集 + 赋值 = 修改; 赋 `NULL` = 删除

筛选

逻辑向量 + 中括号, 是最常用的模式

批量

`apply` 家族替代循环, 管道串起流水线

统一性

运算符也是函数, 函数行为由类型决定

下一问：怎么让 R 自动重复做同一件事？—— 控制流与函数

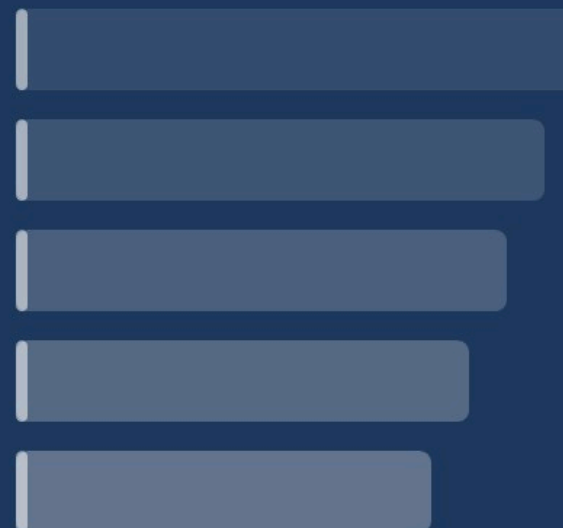
69 / 132

PART 04

控制流与函数

让机器替你重复 —— 这是编程相对于手工操作的根本优势

- if / for / while: 三种控制结构
- 常用函数家族: 数值、字符、统计
- 排序、合并、汇总
- 帮助文档: 遇到不会的函数怎么办
- 自定义函数: 从使用者到创造者
- 调试: 程序出错时怎么办



70 / 132

CONTROL FLOW

三种控制结构

表达式多于一条时，用花括号 {} 包起来

```
if (cond) expr
if (cond) expr1 else expr2

for (var in seq) expr

while (cond) expr

# 示例: 1 到 100 的总和
s <- 0
for (i in 1:100) s <- s + i
```

能量化就量化；不能量化时，循环完全够用

- if 用于分支，for 用于遍历，while 用于条件循环
- R 里 for 循环往往不是最优解 —— 优先考虑向量化
- 但循环胜在直观，初学阶段先求写对、再求写快
- 用 next 跳过本轮，用 break 跳出循环

71 / 132

CONTROL FLOW

控制流实例

两个经典小练习：求和、以及筛出能被 3 整除的数

```
> sum(1:100)
[1] 5050

> x <- 1:100
> sum(x[x %% 3 == 0])
[1] 1683
```

向量化写法：sum(1:100) 比手写 for 循环更快、更短

`x[x %% 3 == 0]` 是「向量化过滤」的标准模式

`%%` 是取模运算符，`x %% 3 == 0` 表示能被 3 整除

结果 $1683 = 3 + 6 + \dots + 99$ ，共 33 项

能向量化就不写循环 —— 代码短，出错少，跑得还快

72 / 132

FUNCTIONS

数值函数速查

这些函数几乎每天都会用到

函数	作用	示例
<code>abs(x)</code>	绝对值	<code>abs(-3) → 3</code>
<code>sqrt(x)</code>	平方根	<code>sqrt(16) → 4</code>
<code>ceiling(x)</code>	向上取整	<code>ceiling(3.475) → 4</code>
<code>floor(x)</code>	向下取整	<code>floor(3.475) → 3</code>
<code>round(x, digits)</code>	四舍五入	<code>round(3.475, 2) → 3.48</code>
<code>log(x) / log10(x)</code>	自然对数 / 常用对数	<code>log(1) → 0</code>
<code>exp(x)</code>	e 的 x 次方	<code>exp(1) → 2.718</code>

记不住参数? 在 R 里输入 `?round` 就有完整文档

73 / 132

FUNCTIONS

字符函数速查

处理基因名、样本名、注释表，全靠这些

函数	作用	示例
<code>substr(x, start, stop)</code>	截取子串	<code>substr("abcdef", 2, 4) → "bcd"</code>
<code>grep(pattern, x)</code>	按模式搜索	<code>grep("A", c("b", "A")) → 2</code>
<code>sub(pattern, repl, x)</code>	按模式替换	<code>sub("\\s", ".", "a b") → "a.b"</code>
<code>strsplit(x, split)</code>	按分隔符切分	<code>strsplit("a,b", ",") → list</code>
<code>paste(..., sep)</code>	拼接字符串	<code>paste("x", 1:2) → "x 1" "x 2"</code>
<code>toupper / tolower</code>	大小写转换	<code>toupper("abc") → "ABC"</code>

字符串操作 + 正则表达式，是生信数据清洗的核心技能

74 / 132

正则表达式：速查表

grep() 与 sub() 的 pattern 参数就是正则表达式 —— 这几张表贴在手边就够了

regexlib · Cheat Sheet

按元字符分类的速查表，写 pattern 时随手查

<http://regexlib.com/CheatSheet.aspx>



regex101 · 在线测试

粘一行 pattern 立刻高亮匹配，支持 40+ 种语言方言语法

<https://regex101.com/>



stringr · tidyverse 文档

R 里处理字符串的事实标准，正则只是它能力的一部分

<https://stringr.tidyverse.org/>



RStudio · Strings Cheat Sheet

stringr / stringi / glue 一页纸速查，彩印贴墙

<https://rstudio.github.io/cheatsheets/strings.pdf>



FUNCTIONS

统计与概率函数

R 的四大分布函数族: d / p / q / r

```
dnorm(x)    # 密度函数 density
pnorm(q)    # 累积概率 cumulative
qnorm(p)    # 分位数 quantile
rnorm(n)    # 随机数 random
```

```
> pnorm(1.96)
[1] 0.975
> qnorm(0.9)
[1] 1.281552
```

- d=密度, p=累积概率, q=分位数, r=随机数
- 把 norm 换成 binom / pois / unif 就是其他分布
- set.seed(1234) 让随机数可复现 —— 写论文必须加
- p 和 q 互为反函数: pnorm(qnorm(0.9)) 回到 0.9

记住 d/p/q/r 这个命名规律, 你就掌握了 R 的所有分布函数

76 / 132

FUNCTIONS

排序: order() 与 sort()

这是数据框排序的标准做法

```
> data(mtcars)
# 按 mpg 升序
> mtcars[order(mtcars$mpg), ]

# 按 mpg 升序、cyl 降序
> mtcars[order(mtcars$mpg, -mtcars$cyl), ]

> head(sort(mtcars$mpg))
[1] 10.4 10.4 13.3 14.3 14.7
```

sort() 返回排好序的值, order() 返回排序后的下标

对数据框排序要用 order(): df[order(...),]

变量前加负号就是降序 —— 只对数值型有效

多列排序时, 越靠前的列优先级越高

排序数据框: df[order(df\$col),], 这个模式要背下来

77 / 132

FUNCTIONS

合并: merge()

把两张表按共同的列拼起来

```
> total <- merge(dfA, dfB, by = "ID")

# 按多列合并
> total <- merge(dfA, dfB, by = c("ID", "Country"))

# 保留所有行 (外连接)
> merge(dfA, dfB, by = "ID", all = TRUE)
```

- by 指定按哪些列合并, 列名要一致
- 默认是内连接: 只保留两边都有的行
- all.x / all.y / all 控制左、右、全外连接
- 表达矩阵与样本信息表的合并, 是最常见的操作

合并前先检查: ID 列有没有重复、有没有缺失

78 / 132

FUNCTIONS

汇总：从明细到统计量

按分组变量切分数据，对每组算一个统计量

```
> aggregate(mpg ~ cyl, data = mtcars, FUN = mean)
  cyl      mpg
1   4 26.66364
2   6 19.74286
3   8 15.10000
```

分组汇总 = 把「一大堆数」变成「一张结论表」

- 公式 `mpg ~ cyl` 读作「按 cyl 分组，对 mpg 汇总」
- FUN 可以是 `mean`、`sd`、`length` 或自定义函数
- 这就是 `split-apply-combine` 模式的最简形式
- 更现代的做法是 `dplyr::group_by()` + `summarise()`

79 / 132

汇总 → tidyverse: 把表格玩出花来

两种主流写法: 基础 aggregate vs dplyr 管道

```
# 基础 R: aggregate() 函数
> aggregate(mtcars[, 'mpg'], by=list(mtcars$cyl), FUN=mean)
  Group.1      x
1      4 26.66364
2      6 19.74286
3      8 15.10000

# dplyr: group_by + |> 管道
> library(dplyr)
> group_by(mtcars, cyl) |> summarize(mean=mean(mpg))
# A tibble: 3 × 2
   cyl mean
<dbl> <dbl>
1     4  26.7
2     6  19.7
3     8  15.1
```

|> 是 R 4.1+ 的原生管道, 把前一步的结果作为下一步的第一个参数

group_by + summarize = 可读性极高的 split-apply-combine

dplyr 输出是 tibble (增强版 data.frame), 显示更友好

tidyverse 包含 dplyr / tidyr / ggplot2 / readr / purrr 等理念一致的包

管道的哲学: 代码从上往下读, 就像在读一段自然语言

遇到不会的函数怎么办?

答案永远是：查帮助文档。这是最重要的一个技能

```
> ?lm           # 查函数文档
> example(lm)    # 跑官方示例
> ?? "linear model" # 全文搜索

# 文档结构 (以 lm 为例) :
#   lm(stats)      函数名及所在包
#   Description    这个函数做什么
#   Usage          怎么调用
#   Arguments      每个参数是什么意思
#   Examples       可直接运行的例子
```

?函数名 是最快的入口; ??关键词 做模糊搜索

Examples 部分直接复制到控制台跑一遍，比读文档快得多

看 Usage 里的参数默认值，就知道哪些可以省略

找不到就搜 GitHub 上的源码或用例

不背函数，会查就行 —— 这是 R 使用者最重要的元技能

FUNCTIONS

自定义函数：从使用者到创造者

函数的意义：把「重复的操作」变成「一个名字」

```
> square <- function(n) {  
  n * n  
}  
> square(4)  
[1] 16  
  
# 带默认参数  
> greet <- function(name = "world") {  
  paste("Hello,", name)  
}
```

当你第二次复制同一段代码时，就该把它写成函数了

- function(参数) { 函数体 } 是最基本的写法
- 最后一行表达式的值就是返回值，也可以显式 return()
- 给参数设默认值，可以让调用更简洁
- 函数是 R 里的「一等公民」：可以赋值、传参、返回

82 / 132

FUNCTIONS

参数是怎么匹配的?

R 的参数匹配有三步规则，理解它能避免很多迷惑行为

```
f <- function(a, b = 2, ...) c(a, b)
```

f(1)	# 1 2	按位置
f(a = 1)	# 1 2	按名字
f(1, 3)	# 1 3	按位置
f(b = 5, a = 1)	# 1 5	名字优先
f(1, c = 9)	# 1 2	c 进 ... 被忽略

第一步：完全匹配参数名

第二步：部分匹配参数名前缀（容易出 bug，慎用）

第三步：按位置匹配剩余参数

... 可以接收任意多的额外参数

复杂函数建议全部用名字传参，可读性最好

参数多的时候一律写名字 —— 少打几个字，多花几小时不值

83 / 132

FUNCTIONS

作用域：变量在哪里生效？

理解作用域，才能理解为什么函数里的变量不会污染外面

函数内部先找

函数体里定义的变量，只在这个函数里有效

找不到往外找

找不到就去定义函数的那个环境里找（词法作用域）

找不到再往上

一路往上找，直到全局环境

全局环境

用 `<-` 可以显式往上层赋值 —— 但强烈不推荐

为什么这么设计

函数之间互不干扰，代码才可组合、可测试

写函数时尽量只依赖参数，不要依赖全局变量 —— 否则很难复用

84 / 132

代码一跑就报错

新手第一反应: "这段肯定有问题...是 R 语言的锅!"

你离跑通只剩三步

老手第一反应: 读报错 → 复现 → 二分定位, 三招内 90% 的 BUG 能修

DEBUG

程序报错了怎么办？

读报错信息是编程最重要的技能之一，没有之一

```
> log("a")
Error in log("a") :
  non-numeric argument to mathematical function

> traceback()          # 看调用栈
> options(error = recover) # 出错时进入浏览器
> browser()            # 在函数里手动设断点
```

报错信息分三段：哪一行、调用了什么、错在哪

non-numeric argument 是最常见的一类 —— 类型不对

object not found 表示变量名拼错了或没定义

不要一报错就重装包 —— 先读报错

80% 的答案就写在报错信息里 —— 先读，再搜，最后才问

86 / 132

趣味练习：用随机数估 π

200 行数学史，10 行 R 代码 —— 蒙特卡洛方法

```
estimate_pi <- function(n = 10000) {  
  x <- runif(n, -1, 1)  
  y <- runif(n, -1, 1)  
  inside <- (x^2 + y^2) <= 1  
  4 * mean(inside)  
}  
  
> estimate_pi(100000)  
[1] 3.14296
```

10 行代码算出一个数学常数 —— 这就是编程的乐趣所在

- 正方形面积 4，内接圆面积 π ，比值就是 $\pi/4$
- 在正方形里随机撒点，落在圆内的比例 $\rightarrow \pi/4$
- 点数越多越准 —— 这就是蒙特卡洛方法的直觉
- 同样的思路可以用来做置换检验、Bootstrap

SUMMARY

PART 04 小结

你现在已经能写出「完整的程序」了

控制流

if 分支、for 遍历、while 条件循环

函数库

数值、字符、统计、排序、合并、汇总六大类

元技能

?函数名 + example() —— 会查比会背重要

自定义函数

把重复操作封装成名字，代码从「脚本」变成「工具」

调试

读报错 → traceback → browser，三步定位问题

下一问：算出来的结果怎么存下来、怎么读进来？

88 / 132

数据进出与 workspace

分析不是自娱自乐——数据要读进来，结果要交出去

- 输入: `read.table` / `readLines` / `scan`
- 输出: `write.table` / `writeLines` / `cat`
- `rio`: 一个接口搞定所有格式
- 工作空间: `ls` / `rm` / `save` / `setwd`
- 一个完整的清洗小例子

I/O

把数据读进来

read.table 家族是 R 最基础的数据入口

```
> df <- read.table("data.txt", header = TRUE,
                    sep = "\t", stringsAsFactors = FALSE)

> df <- read.csv("data.csv")

# 只读前几行看看格式
> readLines("data.txt", n = 5)

# 最灵活的读取方式
> scan("data.txt", what = "")
```

header: 第一行是不是列名

sep: 分隔符, 制表符写 \t, 逗号写 ,

stringsAsFactors = FALSE 避免字符列被自动转成因子

readLines(n=5) 先看格式再读 —— 这个习惯能省很多时间

读文件前先看前几行 —— 90% 的读入错误都源于没看格式

90 / 132

I/O

把结果交出去

能交付的结果，才是分析的结果

```
> write.table(df, "out.txt", sep = "\t",  
              quote = FALSE, row.names = FALSE)  
  
> write.csv(df, "out.csv", row.names = FALSE)  
  
> writeLines(c("line1", "line2"), "out.txt")  
  
> cat("result:", 42, "\n")
```

row.names = FALSE 和 quote = FALSE 这两个参数，几乎每次都要加

quote = FALSE 避免字符被加上引号

row.names = FALSE 避免多出一列没用的行号

writeLines 适合输出纯文本，cat 适合打印到终端

yulab.utils::show_in_excel() 可以直接用 Excel 打开数据框

91 / 132

I/O

rio: 数据进出的瑞士军刀

一个 import(), 一个 export(), 自动识别格式

```
> library(rio)

> df <- import("data.xlsx")
> df <- import("data.csv")
> df <- import("data.sav")

> export(df, "out.xlsx")
> export(df, "out.tsv")
```

靠文件后缀自动判断格式, 不用记一堆函数名

支持 Excel、CSV、TSV、JSON、SPSS、Stata、SAS 等

不同来源的数据用同一个接口, 代码更干净

需要精细控制时, 再回到 read.table 家族

日常用 rio 图省事, 需要精细控制时用基础函数

92 / 132

FORMATS

生信常见文件格式速查

看到这些后缀，你应该知道它是什么

格式	内容	常见工具
FASTA	序列数据	seqkit、Biostrings
FASTQ	序列 + 质量值	fastp、FastQC
GFF / GTF	基因组注释	rtracklayer、GenomicFeatures
BED	基因组区间	bedtools、rtracklayer
SAM / BAM	比对结果	samtools、Rsamtools
VCF / BCF	变异信息	bcftools、VariantAnnotation
表达矩阵	count / TPM 表	read.table、DESeq2

格式不是细节 —— 选错格式，后面的分析工具全都不对

93 / 132

PRACTICE

一个完整的清洗小例子

从原始文件到可分析的数据框，一共七步

```
df <- read.csv("expr.csv", row.names = 1)
df <- df[, colSums(df) > 0]      # 去掉全 0 列
df <- df[rowSums(df) > 10, ]    # 去掉低表达行
df[is.na(df)] <- 0             # 缺失填 0
df <- log2(df + 1)              # 对数变换
df <- df[order(-apply(df, 1, sd)), ] # 按方差排序
head(df[, 1:3])
```

■ 每一步都要问：这一步会不会引入偏差？

■ 过滤阈值不能随便定 —— 要有依据

■ 对数变换前要处理 0 值，否则 $\log(0)$ 是负无穷

■ 清洗过程写进脚本，才能被别人复现

数据清洗没有标准答案，但必须每一步都能解释为什么

94 / 132

WORKSPACE

工作空间：管好你的现场

环境里的对象就是你的全部工作成果 —— 别让它变成一团乱麻

`ls()`

列出当前环境里所有对象

`rm(x)`

删除对象；`rm(list = ls())` 清空全部

`save.image()` / `load()`

保存 / 恢复整个工作空间 (.RData)

`getwd()` / `setwd()`

查看 / 设置工作目录

`sink()`

把控制台输出重定向到文件

一条建议

不要保存 .RData —— 让脚本从原始数据重新跑一遍

最好的工作空间是「空的」：一切从脚本重建，而不是从上次的现场接着跑

95 / 132

SUMMARY

PART 05 小结

数据能进能出，分析才算闭环

进

read.table 家族 + rio::import()

出

write.table 家族 + rio::export()

现场

ls / rm / getwd / setwd, 但别依赖 .RData

习惯

先看格式再读，输出时去掉行号和引号

清洗

每一步都要能解释为什么 —— 这是科学要求

下一问：怎么把代码、结果和图，打包成一份能交的报告？

96 / 132

PART 06

R Markdown & Quarto 可重复报告

代码、文字、图、公式在同一个文件里 —— 一键生成报告

- 为什么要可重复报告
- Markdown 语法：十分钟学会
- 代码块与图表参数
- YAML 与参数化报告
- Quarto：跨语言多输出的下一代工具
- `render()` + `for`：批量出报告

97 / 132

WHY

为什么要 R Markdown?

因为「把图复制到 Word 里」这种做法，是不可复现的

问题一

图是手动截的，数据和代码对不上，改一个参数要全部重做

问题二

报告里的数字是手打的，改数据后忘了改文字，结果自相矛盾

问题三

审稿人要原始分析过程，你只能回忆半年前点了什么

R Markdown 的答案

文字、代码、图、结果写在一个 .Rmd 文件里，一键渲染

额外好处

这份 .Rmd 就是实验记录，交作业、投稿附录都能直接用

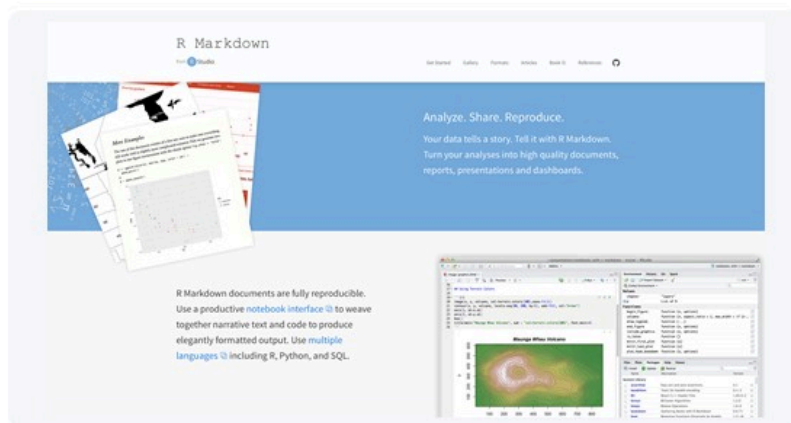
可复现报告的三个要素：数据、代码、叙述，在同一个文件里

98 / 132

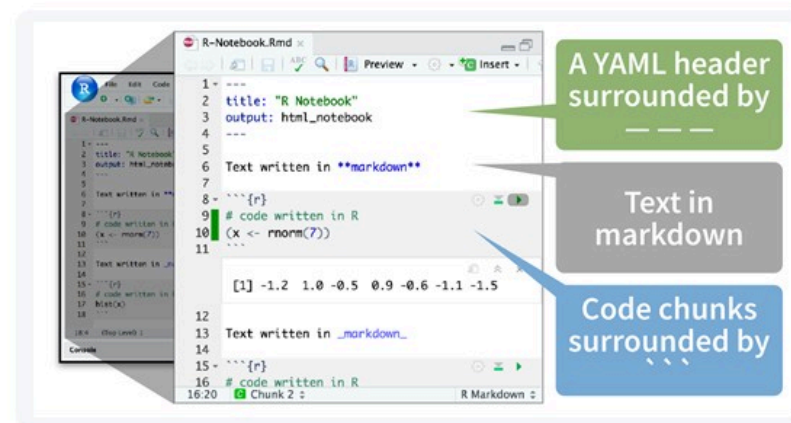
WORKFLOW

R Markdown 的工作流

从 .Rmd 到最终报告, 只需要一个 render()



Rmd → 渲染 → 多种输出格式



代码块、叙述、结果在同一文档中交织

一次编写, 可以输出 HTML / PDF / Word / 幻灯片

99 / 132

MARKDOWN

Markdown 语法：十分钟学会

Markdown 的目标是「让写作者专注于内容，而不是排版」

一级标题, ## 二级标题

多个 # 表示更低的层级

正文直接写, 空行分段

****粗体****、**斜体**、``代码``

- 无序列表, 1. 有序列表

列表项前加符号或数字

[链接文字](网址)

![图片说明](图片路径)

\$公式\$ 与 \$\$块级公式\$\$

行内用单美元, 独立成行用双美元

代码块 (三个反引号 + {r})

代码块会被真正执行, 结果自动插入

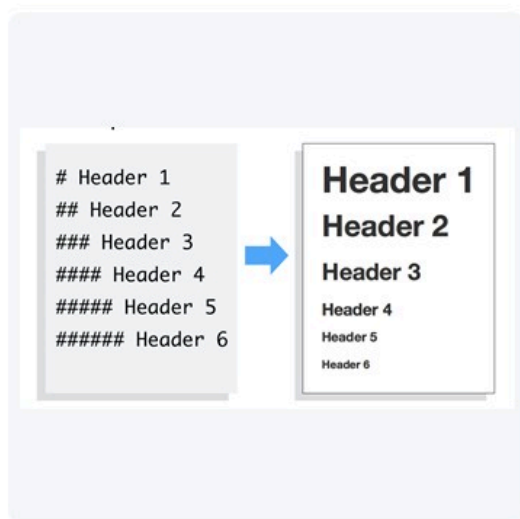
Markdown 被 GitHub、StackOverflow、Reddit 等大量网站采用

100 / 132

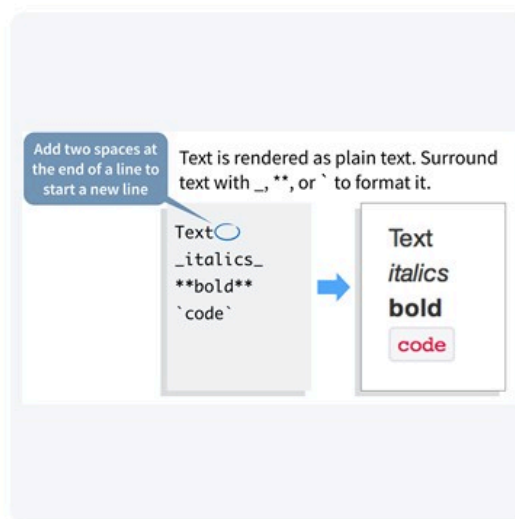
MARKDOWN

Markdown 实例

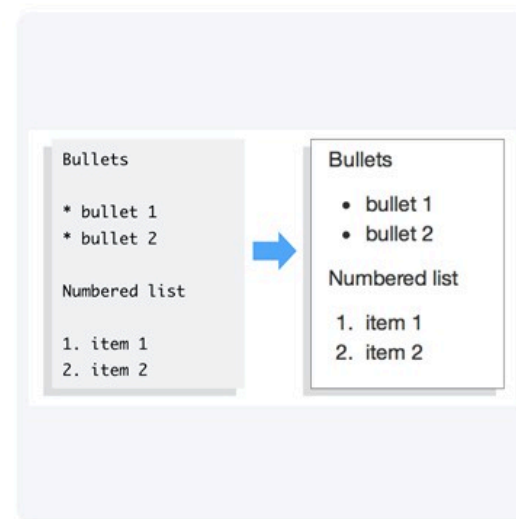
基础语法示例，亲手跑一遍就记住



标题：用 # 的数量控制层级



正文：粗体、斜体、行内代码



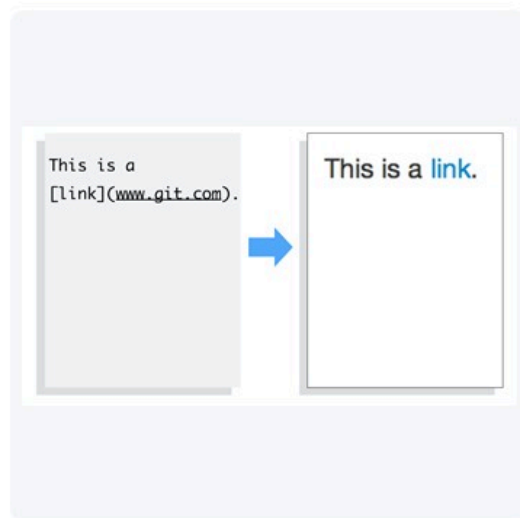
列表：无序与有序

101 / 132

MARKDOWN

Markdown 进阶

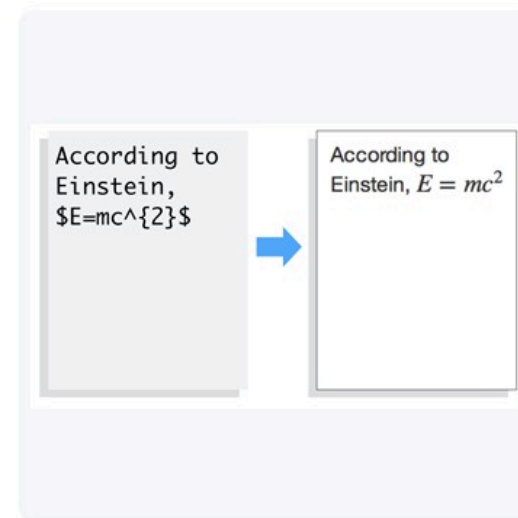
链接、图片、公式、代码块



超链接: [文字](网址)



图片: ![说明](路径)



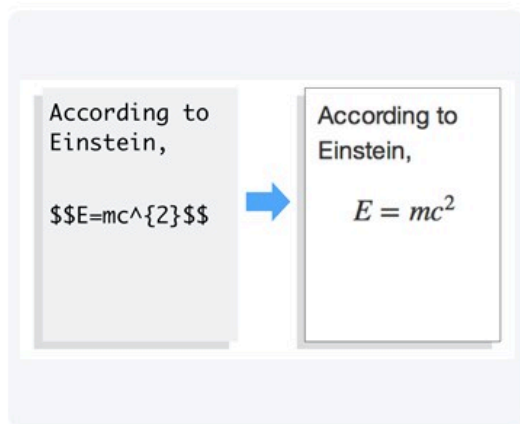
行内公式: `$公式$`

102 / 132

MARKDOWN

代码块与块级公式

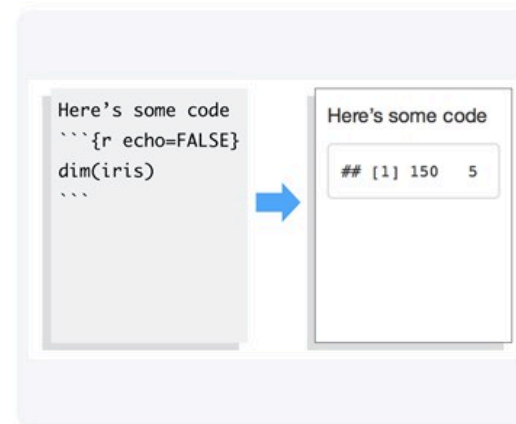
这是 R Markdown 区别于普通 Markdown 的地方



块级公式: `$$公式$$`



代码块: 三个反引号 + `{r}`



echo 控制是否显示代码

代码块是 R Markdown 的核心 —— 它让文档「活」起来

103 / 132

代码块参数

几个参数就能控制代码与结果的显示方式

```
```{r, echo = FALSE}
echo=FALSE: 只显示结果, 不显示代码
summary(mtcars$mpg)
```

```{r, fig.height = 4, fig.width = 6}
plot(mtcars$wt, mtcars$mpg)
```
```

把 `echo = FALSE` 用在最终报告里, 读者只看结论不看实现

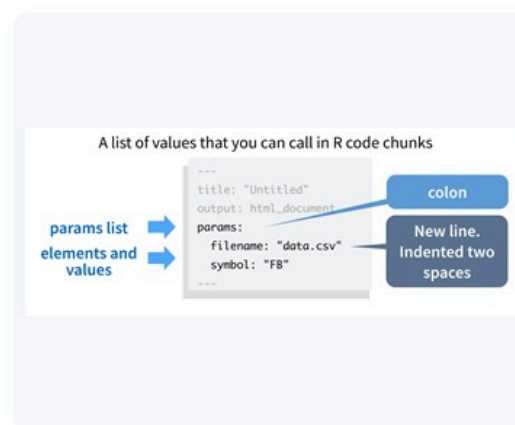
- echo: 是否显示代码
- fig.width / fig.height: 图的大小 (英寸)
- eval: 是否执行代码
- warning / message: 是否显示警告和提示信息

YAML 头部与参数

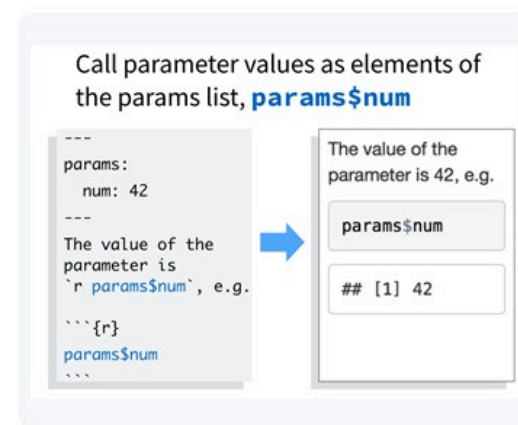
文件开头用 --- 包起来的部分，控制整篇文档的元信息



YAML: 标题、作者、输出格式



parameters: 声明可变参数



在正文中引用 params

参数化报告：同一份 .Rmd，传不同参数就生成不同样本的报告

RMD

render() + for: 批量出报告

一次编写，为 20 个样本各出一份报告

```
library(rmarkdown)

for (s in c("sample1", "sample2", "sample3")) {
  render("report.Rmd",
        output_file = paste0(s, ".html"),
        params = list(sample = s))
}
```

- render() 可以在脚本里调用，实现自动化
- output_file 指定输出文件名
- params 把参数传进 Rmd 的 parameters 段
- 这是生信分析中最实用的自动化模式之一

给 100 个样本出报告，不再需要复制 100 次文件

106 / 132

TEMPLATE

一份完整的报告骨架

以后交作业可以直接从这里开始

```
---
title: "差异表达分析报告"
author: "你的名字"
output: html_document
---

## 1. 数据概览
```{r}
dim(expr)
```

## 2. 结果
```{r, echo=FALSE}
head(res)
```
```

- YAML 头部 → 叙述 → 代码块 → 结果，交替出现
- 每一节讲一个明确的问题
- 代码块和它的解读要挨在一起 —— 读者不用来回翻
- 最后一节放 `sessionInfo()`，记录运行环境

报告的骨架是：问题 → 数据 → 方法 → 结果 → 结论

107 / 132

NEXT-GEN

Quarto: 下一代科学出版工具

由 Posit (RStudio 团队) 出品, R Markdown 的继任者

跨语言

原生支持 R / Python / Julia / Observable JS, 一个 Qmd 文件里混合多种语言

多输出格式

HTML / PDF / Word / PPTX / 书籍 / 博客 / 网站 / 期刊投稿模板, 一次编写到处输出

更现代的语法

统一的 YAML 配置、代码块选项; 原生支持交叉引用、文章章节编号

脱离 RStudio

命令行工具 quarto-cli 可独立运行, 配合 VS Code / Neovim 等任何编辑器

学习成本极低

会 R Markdown 就会 Quarto —— 绝大多数语法完全兼容

quarto.org —— 新项目推荐直接用 Quarto, R Markdown 项目继续维护也没问题

108 / 132

COMPARISON

R Markdown vs Quarto: 主要区别

绝大多数 Rmd 语法可以直接拷到 qmd 里用，主要差异在进阶特性

| 维度 | R Markdown (.Rmd) | Quarto (.qmd) |
|-------|--------------------------------------|--|
| 语言支持 | R 为主，通过 reticulate 间接支持 Python | 原生: R / Python / Julia / OJS / Mermaid |
| 输出格式 | HTML / PDF / Word / ioslides 等 ~10 种 | HTML / PDF / DOCX / PPTX / Book / Blog / Website / Journal Templates |
| 运行环境 | 依赖 R 环境 + rmarkdown 包 | 独立 CLI 工具 quarto-cli, R/Python 按需调用 |
| 交叉引用 | 需 bookdown 扩展 | 原生支持 @fig-id / @tbl-id / @sec-id |
| 编辑器支持 | RStudio 最佳 | RStudio / VS Code (Quarto Extension) / Neovim / JetBrains 等 |

两者不冲突: 老项目维持 Rmd, 新项目直接用 Quarto

109 / 132

SUMMARY

PART 06 小结

R Markdown 是你交给老师的作品，也是交给未来自己的记录

一个文件

文字 + 代码 + 图 + 结果，全部在一起

一键渲染

HTML / PDF / Word / 幻灯片，按需切换

Quarto

跨语言多输出的下一代工具，新项目优先

参数化

同一份模板，批量生成不同样本的报告

意义

报告可复现 —— 改数据重跑一遍，结论自动更新

从今天起，实验报告不要再手打数字了

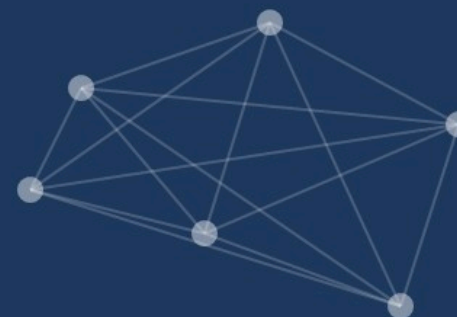
110 / 132

PART 07

从用包到写包

当你开始重复自己的代码，就该考虑把它变成包了

- 三种包来源：CRAN / Bioconductor / GitHub
- 镜像与更新
- 写包的十条经验
- 包的文件结构与文档、测试
- 一个真实案例



111 / 132

PACKAGES

装包：三个来源，三种命令

搞混来源是新手最常见的报错原因

```
# 1. CRAN: 最通用
> install.packages("ggplot2")

# 2. Bioconductor: 组学专用
> install.packages("BiocManager")
> BiocManager::install("clusterProfiler")

# 3. GitHub: 开发中的最新版
> remotes::install_github("YuLab-SMU/treeio")
```

记牢：CRAN 用 `install.packages`, Bioconductor 用 `BiocManager`

Bioconductor 包不要用 `install.packages()` 装 —— 版本会不匹配

GitHub 包往往比 CRAN 版本新，但可能不稳定

装不上先看报错：多半是依赖包、编译器或网络问题

国内记得先设镜像，否则会卡在下载

112 / 132

PACKAGES

镜像与更新

让装包快起来，让包保持最新

设镜像

`options(repos = ...)` 指向清华或中科大镜像 (mirrorselect 图形选择)

检查版本

`check_r()` 检查 R 及 Bioconductor 版本是否需要升级

一键更新

`update_all(which = "CRAN")` 一键更新所有 CRAN 包

分类更新

`which` 参数可取 CRAN / BioC / github, 分开管理三类来源

注意事项

更新前想清楚：有些老代码依赖旧版本，更新后可能跑不通

```
rvcheck: R/Package Version Check
Check latest release version of R and R package (both in 'CRAN', 'Bioconductor' or 'Github').

Version: 0.2.1
Depends: R (>= 3.4.0)
Imports: BiocManager, utils, yaml, utf8
Published: 2021-10-22
DOI: 10.32614/CRAN.package.rvcheck
Author: Guangchuang Yu [aut, cre], Chun-hui Gao [ob]
Maintainer: Guangchuang Yu <guangchuangyu@gmail.com>
License: Artistic-2.0
URL: https://github.com/GuangchuangYu/rvcheck
NeedsCompilation: no
Materials: NEWS
CRAN checks: rvcheck.results

Documentation:
Reference manual: rvcheck.html, rvcheck.pdf

Downloads:

mirrorselect: Test CRAN/Bioconductor Mirror Speed
Testing CRAN and Bioconductor mirror speed by recording download time of 'src/base/COPYING' (for CRAN) and 'packages/rdense/bioc/html/ggnetc.html' (for Bioconductor).

Version: 0.0.3
Depends: R (>= 3.4.0)
Imports: utils, yaml, utf8 (>= 0.1.7)
Published: 2024-08-29
DOI: 10.32614/CRAN.package.mirrorselect
Author: Guangchuang Yu [aut, cre]
Maintainer: Guangchuang Yu <guangchuangyu@gmail.com>
License: Artistic-2.0
NeedsCompilation: no
CRAN checks: mirrorselect.results

Documentation:
Reference manual: mirrorselect.html, mirrorselect.pdf

Downloads:
```

「包不存在」多半不是镜像的锅，先确认包名和来源对不对

113 / 132

WHY WRITE

为什么要写 R 包?

不是为了发文章，而是为了让自己的代码活得久一点

避免裹脚布般的脚本

一个 800 行的脚本，改到第三个月就没人敢动了

便于管理与分发

函数、文档、测试、数据各归其位

强迫自己模块化

以功能为单位组织代码，自然就封装成了函数

顺带把文章发了

很多 R 包就是研究的副产品 —— 例如 ggtree、clusterProfiler

让别人能重复你的结果

这是科学的基本要求



写包的本质：把自己的分析流程，变成别人也能用的工具

114 / 132

TEN LESSONS

写包的十条经验（上）

这些经验来自真实的包开发历程，不是教科书条目

1. 写包要趁早

别等脚本写到 1000 行才重构。越早封装，后面越省力

2. 用包组织代码

R/ 放函数，man/ 放文档，tests/ 放测试，data/ 放数据

3. 从痛点出发

没有软件能实现就自己写；现有工具用着不爽就改进它

4. 函数要短

长度控制在一屏之内。超出一屏就要分而治之

5. 刻意练习

代码需要反复阅读和修改。一个包往往要迭代好几年

痛点是最好的选题来源：你自己的不方便，往往是一群人的不方便

115 / 132

TEN LESSONS

写包的十条经验（下）

从能跑，到好用，到被依赖

6. 优化要基于测量

先 profile 再优化。for 循环并不一定比 apply 慢

7. 设计要通用

给自己写代码，顺道把文章发了。站在更高抽象层次思考

8. 对用户友好

统一、抽象、可组合、符合直觉 —— 隐藏内部细节

9. 写好文档

man 文档用 roxygen2，教程用 vignette，长文用 bookdown

10. 长期维护

修正 bug、扩展功能、丰富文档。软件靠谱，用户才会依赖

把东西做大做复杂很容易，难的是反过来 —— 这需要勇气

116 / 132

STRUCTURE

一个 R 包长什么样？

标准目录结构 —— 每个文件夹都有明确职责

R/

所有函数的源代码，一个函数一个文件，文件名通常与函数名一致

man/

帮助文档，由 roxygen2 从代码注释自动生成，不要手写

tests/

单元测试，用 testthat 写，保证改动不破坏旧功能

data/

包自带的示例数据集，用 .rda 格式

DESCRIPTION

包的元信息：名字、版本、作者、依赖

NAMESPACE

导出哪些函数、导入哪些依赖（也由 roxygen2 生成）

记住这个结构，你就能读懂任何一个 R 包的源码

117 / 132

DOCUMENT

用 roxygen2 写文档

文档写在函数旁边，自动生成 man/ —— 这是 R 包的标准做法

```
#' Square a number
#'  
#' @param n A numeric vector.  
#' @return The squared value of n.  
#' @examples  
#' square(4)  
#' @export  
square <- function(n) {  
  n * n  
}
```

- 以 #' 开头的注释就是文档
- @param 描述参数，@return 描述返回值
- @examples 里的代码会被真正执行检查
- @export 表示这个函数对用户可见

文档和代码放在一起，就不会出现「代码改了文档忘了改」

118 / 132

TEST

用 testthat 写测试

测试不是为了证明你对，是为了在改代码时及时发现你错了

```
library(testthat)

test_that("square works", {
  expect_equal(square(2), 4)
  expect_equal(square(0), 0)
  expect_equal(square(-3), 9)
  expect_error(square("a"))
})
```

- expect_equal 判断值相等
- expect_error 判断会不会报错
- 每次改代码后跑一遍测试，就知道有没有破坏旧功能
- 测试写得越早，后面越省心

有测试的包才敢改 —— 这是长期维护的前提

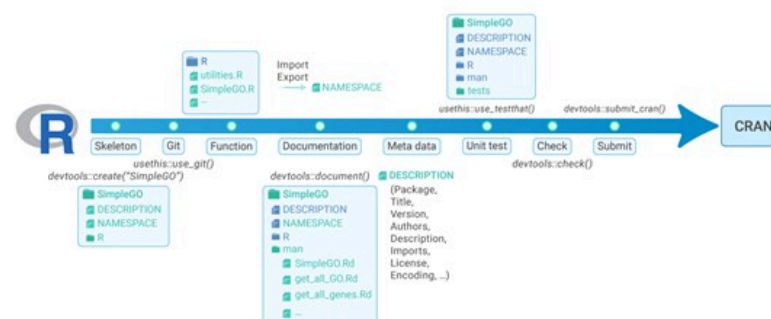
119 / 132

CASE STUDY

案例：一个包是怎么长出来的

从「没有软件可用」到「被上千篇论文引用」

- GOSemSim: 首次实现 Wang 提出的图结构算法
- treeio: 解决数据格式不兼容、没有软件可解析的问题
- seqcombo: 基因组重排示意图原来只能靠手工画
- clusterProfiler: 原有富集分析不支持微生物、不支持多条件比较



Ma XK#, Yu Y#, ... Yu G*, Yang L*. Bioinformatics Software Development.
The Innovation Life. 2024 120 / 132

CASE STUDY

包开发全过程

以 simpleGO 为例：从想法到发布

- 建目录：`usethis::create_package()`
- 写文档：roxygen2 注释
→ 自动生成 man
- 检查与提交：R CMD check 通过后提交

- 写函数：R/ 目录，一个函数一个文件

- 写测试：testthat，保证改动不破坏旧功能

ages specifically designed for the processing and visualization of omics data.¹ Of note, R language has an active community that continues to create new R packages, expanding its applications to diverse conditions and circumstances. In this scenario, it is useful to learn how to create an R package. As for personal requirement, organizing code into an R package, enhancing documentation, testing the code, and improving user-friendliness can greatly benefit future work. For instance, the creation of *clusterProfiler* initially aimed to compare enrichment analysis results from various cell cycle proteomics data.^{2,3} At that time, no tools supported the aggregation and comparison of results under multiple conditions. Similarly, the development of *ggtree* aimed to integrate and visualize phylogenetic trees and related data for joint presentation from an evolutionary perspective.⁴ The eventual application scenarios exceeded the authors' initial expectations, which benefits others' research as well.

To start R package development, several necessary packages, including *usethis*, *roxygen2*, *testthat*, and *devtools*, are required (Table 1). Among them, *usethis* automates a series of settings for creating R packages; *roxygen2* supports literate programming, resulting in the direct embedding of documentation language in the code; *testthat* package is used to write test scripts to ensure the correctness of functions; *devtools* is a tool package for R development and management, which can simplify many development tasks.

We can use `devtools::create("package name")` to create the frame of an R package, which will generate the *DESCRIPTION* and *NAMESPACE* files and an *R* folder. *DESCRIPTION* is the core part of an R package, which includes the key metadata of the package. The *NAMESPACE* file contains information about exported and imported functions. The *R* folder is used to store source codes of the functions during software development, and functions are

usually stored in different files according to the logic of the R code. R code is the most critical because the main purpose of an R package is to write functions to solve specific problems or requirements. In addition to this, the R package will also include a *man* folder for storing documentation, containing package documentation, function documentation and data documentation.

Typically, it is recommended to use `git`, initialize by the `usethis::use_git()` function, for version control during software development. Running `git` remotely can establish a link between the local repository and the remote repository, allowing one to synchronize local development records with the remote repository, usually on a Git code hosting platform. Next, *clusterProfiler* is applied to illustrate the development process and standards of an R package that enables GO enrichment analysis.

The code in the R package needs to be defined in the form of functions and stored in files with the *R* extension. Here, an illustrative package structure, named *simpleGO* (<https://github.com/YuLab-SMU/simpleGO>) using `devtools::create("simpleGO")` is shown in Figure 4, which implements a simple gene ontology (GO) enrichment analysis function regarding *clusterProfiler*.

GO has three branches, Biological Process (BP), Molecular Function (MF), and Cellular Component (CC).⁵ First, the auxiliary function `get_all_GO()` is defined to obtain all GO IDs belonging to a certain GO branch, and `get_all_genes()` is then defined to obtain all human genes annotated with the corresponding GO branch. Based on this, the main function `simpleGO()` is defined (<https://github.com/YuLab-SMU/simpleGO/blob/main/R/simpleGO.R>).

It accepts three inputs: *gene* is a vector of genes to be tested, such as differentially expressed genes. *GOID* is the selected GO ID by the user, and *background* is the total background genes for testing, usually all genes with

The Innovation Life 2(3): 100083, September 16, 2024 5

Bioinformatics Software Development: principles and future directions.

The Innovation Life. 2024

121 / 132

SUMMARY

PART 07 小结

从「用别人的包」到「别人用你的包」，这是一条真实可行的路

装包

认准来源：CRAN / Bioconductor / GitHub

管包

设镜像、定期更新，但更新前先想清楚风险

写包

从痛点出发，函数要短，设计要通用，文档要全

工程

R/ man/ tests/ data/ + DESCRIPTION + NAMESPACE

维护

包是长期资产 —— 靠谱比功能多更重要

你不需要现在就开始写包，但要知道这条路是通的

122 / 132

AI 辅助编码与 Rho: 从工具到协作

当 AI 能写代码时, 真正稀缺的是「提问」和「判断」的能力

- AI 辅助编码: 把 AI 当副驾, 不是当司机
- Rho: R 语言 AI 工作台 (R-centered Human-AI Orchestration)
- Ask / Plan / Act 三档模式
- 你的判断能力, 才是真正的护城河

工具越强, 问问题越贵 —— 你才是唯一的「最后一道防线」

AI ERA

Ask / Plan / Act: 三档协作模式

从一句话答案到整个工程交付，AI 参与的深度是你决定的

第一档: Ask (问)

把报错信息粘过去、把需求说清楚，让 AI 给第一版方案或解释 —— 30 秒拿到起点

第二档: Plan (拆)

把一个大任务拆成函数清单、数据结构草图，让 AI 按模块实现 —— 你总揽全局

第三档: Act (跑)

让 AI 在工程里批量修改、写测试、生成文档 —— 人负责验收与验证

永远不变的红线

AI 写的每一行代码，你必须能看懂、能修改、能对结果负责

最有用的能力

会把问题描述清楚 (prompt)、会验证 AI 给出的答案是否真的正确

AI 让新手的起点变高，但「能验证、能判断」的能力才是真正的护城河

124 / 132

Rho: 一个 R 语言 AI 工作台

R-centered Human-AI Orchestration —— 一个 agent-native 的 R 桌面工作台

Agent-native

不是传统 IDE 加了个聊天框，而是原生支持 AI Agent 替你执行跨步骤的工程任务：写函数、跑脚本、改文档、写测试

本地优先

原生桌面应用，Windows/macOS/Linux 三端齐发。不依赖 Python，不依赖 Jupyter，也不绑 Electron；启动快、占存小、离线也能用

Rho 功能总览

Ask/Plan/Act 三档模式、四面板工程视图、工作区共享上下文、对比 RStudio/Positron 的差异 https://mp.weixin.qq.com/s/A8PVvNPbC_b89x_HR-99GA

Rho 初体验

5 分钟跑通第一个 R 工程：安装 → 新建工程 → 四面板 → 第一次 AI Ask <https://mp.weixin.qq.com/s/X7tY3J9UPRPboGNbVDIWWQ>

Rho 仓库: github.com/YuLab-SMU/Rho —— 工具在进化，「会写代码」的底层能力不会贬值

125 / 132

PITFALLS

新手最常踩的 8 个坑

提前知道，能省下你好几个通宵

路径带中文或空格

装包会莫名失败。安装路径和工作目录都用纯英文

忘记设镜像

下载卡住或极慢。国内第一件事就是设镜像

因子转数值

`as.numeric(factor)` 拿到的是编号，必须先 `as.character()`

下标从 0 开始

R 从 1 开始数。x[0] 返回空向量，不报错但也不对

用 = 赋值

在函数调用里 = 是传参不是赋值，容易产生难以发现的 bug

NA 污染统计

`mean()` 返回 NA，先检查缺失值，再决定怎么处理

保存 .RData 后接着跑

下次打开环境是脏的。养成从脚本重建的习惯

不查文档就猜参数

?函数名 比搜索引擎快，而且一定准确

这八个坑，每一个都能让新手卡一整天

126 / 132

LEARNING PATH

接下来怎么学？

课程只是起点，真正学会要靠自己写

1 抄

把课件的代码一行行敲进 R 里跑一遍，不要复制粘贴

2 改

换自己的数据，改参数，看结果怎么变

3 写

遇到重复劳动就写成函数，遇到重复函数就打包

4 读

读优秀包的源码，看别人怎么组织代码

5 教

能给别人讲明白，才是真的懂了

这五步不是线性的——你会在「改」和「读」之间反复来回

R 不是学出来的，是写出来的

127 / 132

RESOURCES

资源清单

遇到问题时，按这个顺序找答案

官方文档

cran.r-project.org 与 bioconductor.org —
— 最权威，但不好读

R 帮助系统

?函数名、example()、vignette() —— 最快
的入口

R for Data Science

r4ds.hadley.nz —— 免费在线，数据科学入
门首选

Bioconductor 教程

每个包都有 vignette，跟着跑一遍就入门了

社区

Stack Overflow、RStudio Community、Git
Hub Issues

本课程资料

课程讲义与代码会同步更新，优先以课程材
料为准

遇到报错先读报错信息 —— 80% 的答案就写在里面

128 / 132

GLOSSARY

术语速查

本讲出现的关键词，一句话解释

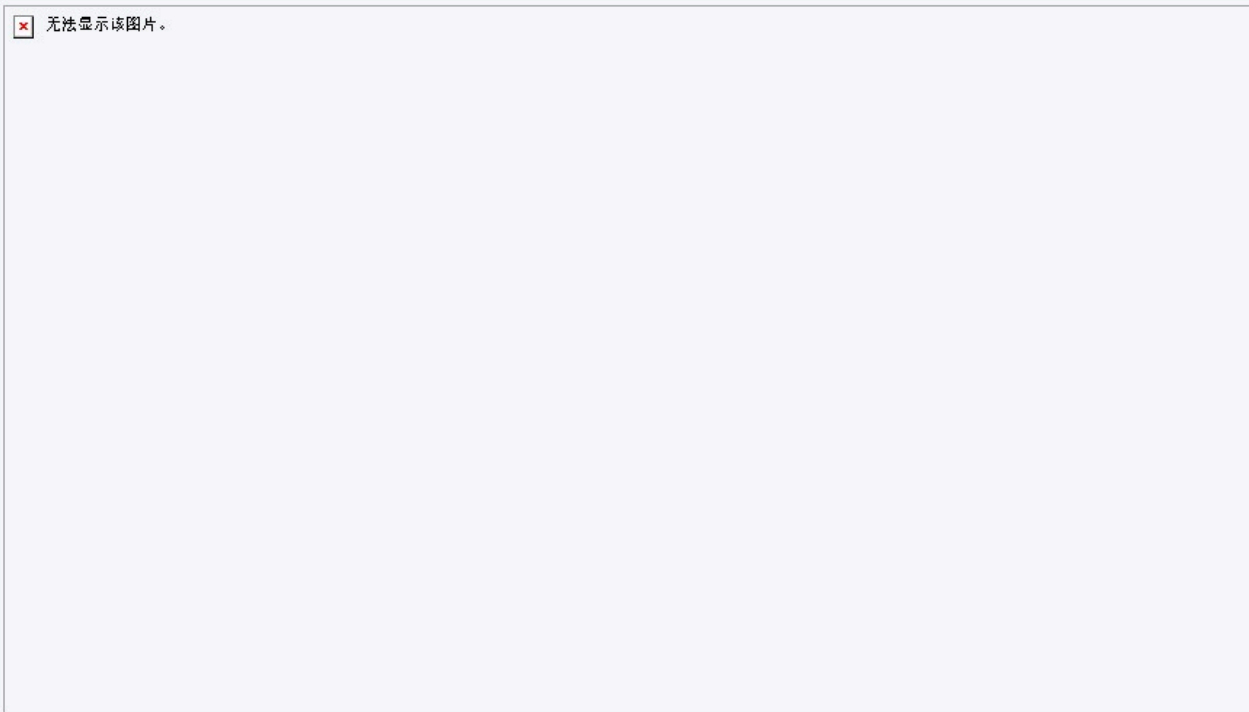
| 术语 | 一句话解释 |
|-----------------|---------------------------------------|
| 向量 (vector) | 同类型元素组成的一维序列，R 的基本单位 |
| 向量化 | 对整个向量逐元素运算，无需写循环 |
| 属性 (attribute) | 附加在对象上的元数据，如 names、dim、class |
| 因子 (factor) | 带顺序的字符型，用于分类变量 |
| 强制转换 (coercion) | 混合类型时自动升级为最宽的类型 |
| 值语义 | 赋值通常产生副本，修改副本不影响原对象 |
| 泛型函数 | 根据参数类型自动选择具体方法的函数 |
| apply 家族 | apply / lapply / sapply / tapply，替代循环 |

术语看不懂时回到这张表 —— 或者用 ?术语 查文档

129 / 132

R 语言第一讲 · 内容纲领思维导图

L1 ROADMAP · MIND MAP



左半：操作与控制（PART00/02/03/04） · 右半：输出与交付（PART01/05/06/07） · 两侧：FRONTIER 前沿 + 收尾进阶

130 / 132

FINAL QUESTION

学完这一讲，你真正获得了什么？

- 不是「记住了 200 个函数」—— 函数会被忘掉，文档随时可查
- 而是：你有了一个把想法变成可执行、可复现流程的工具
- 从此你的分析可以被重跑、被审计、被交接、被改进
- 这才是「会编程」与「会点鼠标」之间的分界线

下一讲：《R 语言在生物信息学数据可视化中的应用》—— 把这些数据变成图

131 / 132

谢谢

R 语言 · 功能基因组学

下一讲：R 语言在生物信息学数据可视化中的应用

请提前装好 R 与 Rho（或 RStudio）

有问题随时提——卡住的时候，问比熬更高效

132 / 132